Disturbance-aware informative planning

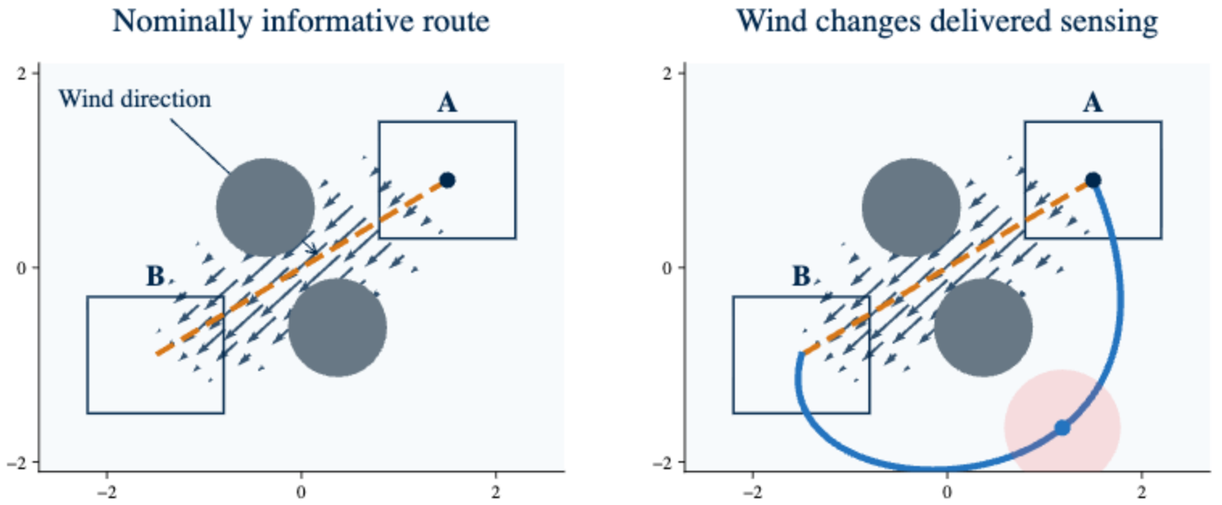
Predicting executed monitoring performance with optimistic planning and real online learning

Kaleb Ben Naveed
Department of Robotics · University of Michigan

Model-based diffusion with a learned disturbance model

An informative plan can lose its value in execution

The central issue is that a nominally informative trajectory may not deliver its predicted monitoring performance when executed under uncertain disturbances.



Dashed orange: nominal route Blue: possible executed motion Arrows: wind

1. Wind-aware planning

Predict how wind and controller intervention change motion, arrival time and sensing.

2. Optimistic planning with online learning

Try favorable plausible alternatives and learn from actual execution.

Robot state, control and spatial disturbance

Generic nominal robot model

$$\dot{x} = f(x, u)$$

Execution under spatial wind

$$\dot{x} = f_d(x, u, \bar{d}(r)), \qquad r = \Psi(x)$$

> **The planner must predict motion under the unknown disturbance.**

Symbol	Meaning
$x \in \mathcal{X} \subset \mathbb{R}^n$	Robot state
$u \in \mathcal{U} \subset \mathbb{R}^m$	Admissible control input
$r \in \mathcal{P}$	Robot position in the coverage space
Ψ	Map from state to position
f	Nominal dynamics
f_d	Dynamics including the disturbance
$\bar{d}(r)$	Fixed, unknown physical wind field

Paper notation: Eq. (4) writes the disturbance-dependent dynamics as $f(x, u, \bar{d}(r))$; here the subscript distinguishes it from the nominal model.

Presentation structure

1 Introduction
Why nominal informativeness can differ from delivered monitoring.

2 Preliminaries
Clarity, POAM, robust policy values and model-based diffusion.

3 Problem formulation
Disturbance-aware monitoring and optimistic performance preference.

4 Method
Algorithm first, then execution prediction, scoring, sampling, and real learning.

5 MBD comparison
The target, importance correction and retained reverse update.

6 Implementation and compute time
Exact settings, measured costs, expensive components, and source files.

2 Preliminaries

Models and numerical tools used by the planner.

Clarity → POAM → robust policy values → model-based diffusion

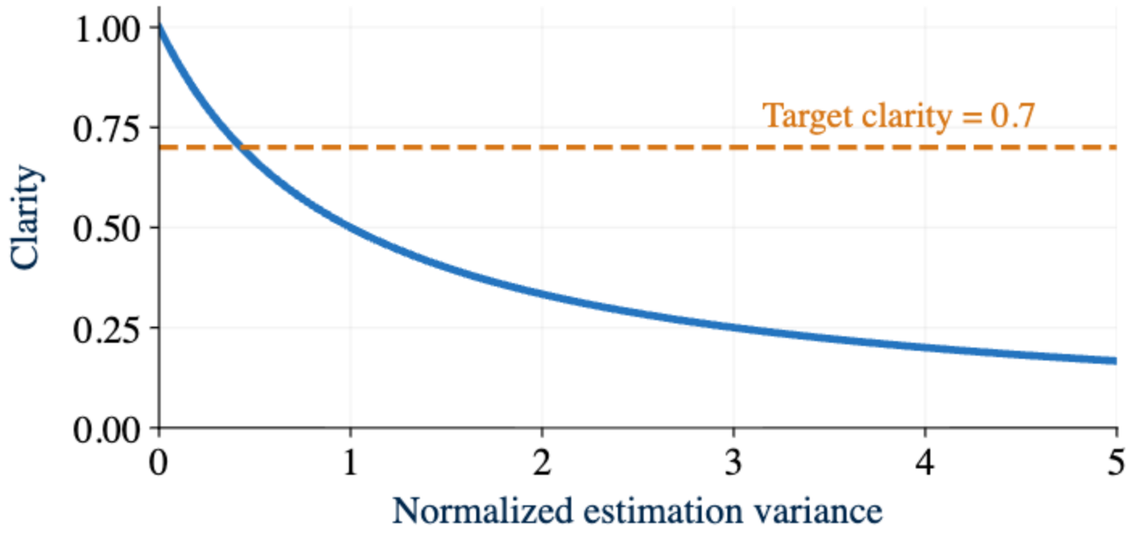
Clarity measures how well a cell is known

Divide the coverage space into N_p cells.

For cell j, estimate a time-varying quantity m_j with normalized variance P_j.

$$q_j = \frac{1}{1 + P_j}, \qquad 0 < q_j \leq 1$$

- $q_j \approx 1$: the quantity is well known.
- $q_j \approx 0$: large estimation uncertainty.
- $\bar{q}_j$: the desired clarity for that cell.



Clarity concerns the monitored quantity.
Wind uncertainty is a separate belief.

Supplied paper: Appendix I, Gaussian clarity. The simulations use scalar independent cell estimates.

Monitoring quality evolves along the executed path

$$\dot{q}_j = \frac{C_j(x)^2}{R}(1 - q_j)^2 - Q_j q_j^2$$

Sensing improves clarity: $C_j(x)$ is the measurement quality at robot state x; $R > 0$ is measurement-noise intensity.

Change degrades clarity: $Q_j \geq 0$ is the process-noise intensity of the monitored quantity in cell j.

$$q_d(q) = \frac{1}{N_p}\sum_{j=1}^{N_p}[\bar{q}_j - q_j]_+, \qquad J = \int_0^T q_d(q(\tau))\, d\tau$$

$[a]_+ = \max(0, a)$; q_d is mean shortfall below target clarity.

$\tau \in [0, T]$ is elapsed prediction time; $T > 0$ is the planning horizon. Lower J means better monitoring.

Wind changes the executed state x, which changes sensing $C_j(x)$ and the task cost J.

Represent the wind that the robot currently knows

At physical time t, the robot has noisy observations of the unknown physical wind field $\bar{d}$:

$$y_m^d = \bar{d}(r_m) + \varepsilon_m^d, \qquad \varepsilon_m^d \sim \mathcal{N}(0, R_d).$$

m: observation index; r_m: its position.
y_m^d: observed wind vector.
ε_m^d: zero-mean measurement noise.

$\mathcal{N}(\mu, \Sigma)$: Gaussian distribution with mean μ and covariance Σ.
R_d: measurement-noise covariance.

The **wind belief** $\mathcal{B}_t^d$ summarizes what those observations imply:

$$\mathcal{B}_t^d = \{\mu_d(\cdot, t), \Sigma_d(\cdot, \cdot, t)\}.$$

$\mu_d(r, t)$: predicted wind mean at position r.
$\Sigma_d(r, r', t)$: covariance between wind at r and r'.

r': a second position; $\cdot$: a position argument.
The label d denotes wind; it is not a power.
Covariance expresses remaining uncertainty.

POAM — Probabilistic Online Attentive Mapping — updates this spatial belief as observations arrive.

Chen, Liu and Khardon, POAM, RSS 2024. The next slides explain one scalar wind component; vector components are modeled separately here.

POAM uses spatial correlation to connect observations

For one scalar wind component, use a **Gaussian process (GP)**: any finite collection of wind values has a joint Gaussian distribution.

$$d(r) \sim \mathcal{GP}(0, k_\vartheta(r, r')) \, .$$

$d(r)$: uncertain wind component at r.
0: prior mean, before observations.
$k_\vartheta(r, r')$: prior covariance between positions.

ϑ: kernel parameters.
The kernel controls how an observation at one position influences predictions elsewhere.

The **attentive kernel** combines spatial correlation lengths:

$$k_\vartheta(r, r') = \sigma_f^2 \left[\bar{w}(r)^\mathsf{T} \bar{w}(r')\right] \sum_{b=1}^{B} \bar{w}_b(r) \bar{w}_b(r') \, \exp\left(-\frac{\|r - r'\|_2^2}{2\ell_b^2}\right) .$$

B: number of kernel components; b: component index.
$\ell_b > 0$: correlation length of component b.
σ_f^2: prior variance scale.

$\bar{w}_b(r)$: normalized component weight at r.
T: transpose; $\| \cdot \|_2$: Euclidean norm.
Larger ℓ_b links more distant positions.

Chen, Liu and Khardon, POAM, Sec. III-B. The following slide defines the location-dependent weights.

What the attentive weights mean

At each position, a neural network produces a nonnegative vector $w_\vartheta(r) \in \mathbb{R}^B$. Normalize it to obtain

$$\bar{w}(r) = \frac{w_\vartheta(r)}{\|w_\vartheta(r)\|_2}, \qquad \bar{w}_b(r) = \text{component } b \text{ of } \bar{w}(r).$$

Which length scales apply here?

Large $\bar{w}_b(r)$ gives component b more influence at position r.

How similar are the two regions?

$\bar{w}(r)^\mathsf{T}\bar{w}(r')$ reduces correlation when their weight vectors differ.

Example with two components: one long correlation length describes smooth wind; a shorter one describes more rapid spatial variation. The weights can favor different components in different regions.

> **These are kernel weights over spatial length scales. Later, MBD uses different weights over sampled decisions.**

$\mathbb{R}^B$: vectors with B real entries. ϑ includes the network and kernel parameters. The numerical visual uses specified illustrative weights.

POAM predicts wind through a sparse Gaussian posterior

Choose M representative **inducing positions** $Z = (z_1, \dots, z_M)$. They are model locations, not additional observations. Collect their wind values in $d_Z = [d(z_1), \dots, d(z_M)]^\mathsf{T}$.

$$d_Z \sim \mathcal{N}(m_Z, S_Z), \qquad K_{ZZ} = k_\vartheta(Z, Z), \qquad \beta(r) = K_{ZZ}^{-1} k_\vartheta(Z, r).$$

m_Z: mean vector of the M inducing values.
S_Z: their $M \times M$ posterior covariance.
K_{ZZ}: prior covariance matrix at Z.

$k_\vartheta(Z, r)$: vector of covariances from Z to r.
$\beta(r)$: interpolation-coefficient vector.
K_{ZZ}^{-1}: matrix inverse (computed by a solve).

$$\mu_d(r) = \beta(r)^\mathsf{T} m_Z$$

$$\Sigma_d(r, r') = \underbrace{k_\vartheta(r, r') - \beta(r)^\mathsf{T} K_{ZZ} \beta(r')}_{\text{conditional residual}} + \beta(r)^\mathsf{T} S_Z \beta(r').$$

The residual retains uncertainty that the finite inducing values cannot represent. Local wind standard deviation is $\sigma_d(r) = \sqrt{\Sigma_d(r, r)}$.

POAM, Eq. (4). Time t is suppressed on these slides: every posterior is the current one.

How a new observation changes the POAM posterior

At r_m, observe y_m^d. For this scalar component, the measurement-noise variance is $\sigma_n^2 > 0$ (the scalar counterpart of R_d). Hold the kernel and inducing positions fixed.

$$\beta_m = \beta(r_m), \qquad g_m = \frac{S_Z \beta_m}{\sigma_n^2 + \beta_m^\mathsf{T} S_Z \beta_m}.$$

β_m: interpolation coefficients at the observation.
g_m: gain vector, determining how much each inducing mean responds.

$y_m^d - \beta_m^\mathsf{T} m_Z$: **innovation**, the measured wind minus its predicted mean.
Superscript $+$ means after this observation.

$$m_Z^+ = m_Z + g_m\left(y_m^d - \beta_m^\mathsf{T} m_Z\right), \qquad S_Z^+ = S_Z - g_m \beta_m^\mathsf{T} S_Z.$$

Mean

A nonzero innovation changes the wind estimate. A zero innovation leaves it unchanged.

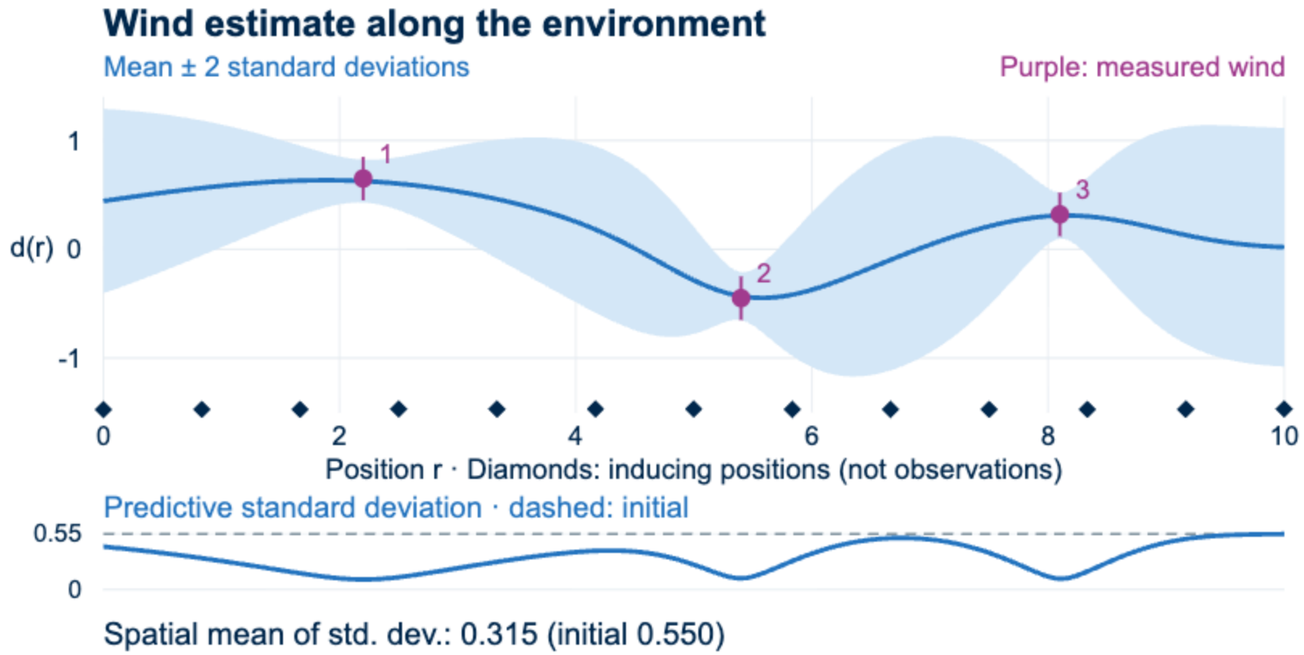
Covariance

Uncertainty decreases in correlated inducing values. Recompute μ_d and Σ_d from m_Z^+, S_Z^+.

Fixed-representation sparse variational Gaussian update; POAM Eqs. (7)–(8), (19)–(21). Full online POAM can also refit its kernel and relocate inducing positions.

Watch POAM learn a spatial wind model

Three observations assimilated



1. Start from a spatial prior
2. Receive one wind observation
3. Update the sparse posterior
4. Predict at an unobserved position
5. Assimilate a second observation
6. **Assimilate a third observation**

After three observations, the posterior retains uncertainty between observations. The short-scale region on the right needs more local evidence.

$$\mathcal{B}_3^d = \mathrm{POAMUpdate}(\mathcal{B}_2^d; r_3, y_3^d)$$

$\mathcal{B}_n^d$: belief after n observations; $\sigma_d(r)$: wind standard deviation. The numerical example fixes the kernel and inducing positions; wind uses normalized units.

Why position alone is not enough

To predict a future constraint violation, we must specify both the dynamics and the future control inputs.

Consider a robot moving toward a wall. Let p be its position, with the wall at $p = 0$.

$$h(x) = p, \qquad h(x) \leq 0 \text{ is the permitted side.}$$

Two robots at $p = -1$ have the same h, but a faster robot needs more distance to stop.

A geometric constraint

$h(x)$ tests the current state.

A policy value

Tests the largest future value of h when a specified control law is followed.

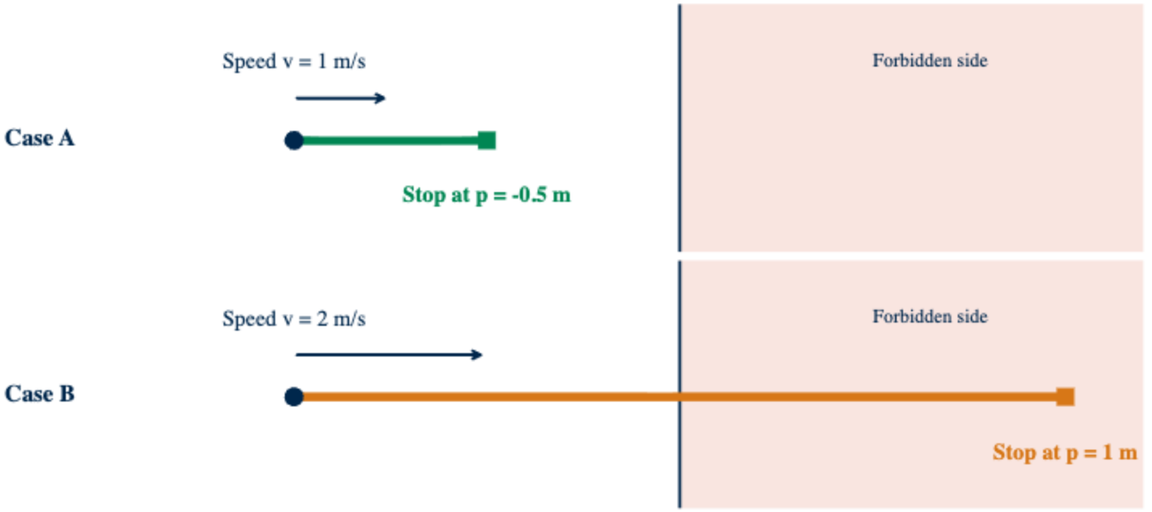
That specified control law is the design policy, denoted π_D. It makes the future-motion test well defined.

Policy-value construction: Knoedler et al., Safety on the Fly, Sec. III-A.

Example: evaluate the policy “brake until stopped”

Illustrative model: $x = (p, v)$, $\dot{p} = v$, $\dot{v} = u$, initially $v \geq 0$. Use $u = -a_b$ until $v = 0$, then $u = 0$, where $a_b > 0$ is braking acceleration.

$$V(x) = \max_{\tau \geq 0} p(\tau) = p + \frac{v^2}{2a_b}$$



Case A: $p = -1$, $v = 1$, $a_b = 1$.
$V = -0.5$: braking stops before the wall.

Case B: $p = -1$, $v = 2$, $a_b = 1$.
$V = 1$: the braking trajectory crosses the wall.

> **The value depends on the chosen policy. Wind, actuator limits or a different braking law change this prediction.**

From the example to a robust policy value

Let $\phi_D(\tau; x, d)$ be the state reached from x after time τ when following π_D under disturbance signal d.

$$V_a(x) = \sup_{d \in \mathscr{D}} \sup_{\tau \geq 0} h_a(\phi_D(\tau; x, d))$$

h_a: constraint function for constraint a; violation means $h_a > 0$.
$\mathscr{D}$: disturbance signals considered in the source formulation.

Inner supremum: largest constraint value over time.
Outer supremum: largest such value over disturbances.

$V_a(x) \leq 0$
All trajectories included in this test satisfy constraint a.

$V_a(x) > 0$
At least one trajectory in the test violates the constraint under this policy.

RPCBF = robust policy control barrier function. Knoedler et al., Eqs. (6), (10). Their numerical construction uses a finite horizon, sampled disturbances and spline maxima; guarantees require additional conditions.

A policy value can guide the applied input

The task policy requests u_{nom}. The execution controller chooses the input u closest to it while meeting a value-rate condition:

$$\min_{u \in \mathcal{U}} \frac{1}{2}\|u - u_{\text{nom}}\|^2 \quad \text{subject to} \quad \nabla V_a(x)^{\mathsf{T}} f_d(x, u, d) + \kappa V_a(x) \leq 0.$$

The condition is imposed for every constraint a and every disturbance d included in the controller's test.

$\mathcal{U}$: actuator input set.
∇V_a: sensitivity of the value to state.
$\kappa > 0$: controller value-rate coefficient.

If u_{nom} satisfies the conditions, it is applied unchanged.
Otherwise, the controller selects a different input.

> **This changes motion and sensing. Our planner predicts that controller response when evaluating monitoring performance.**

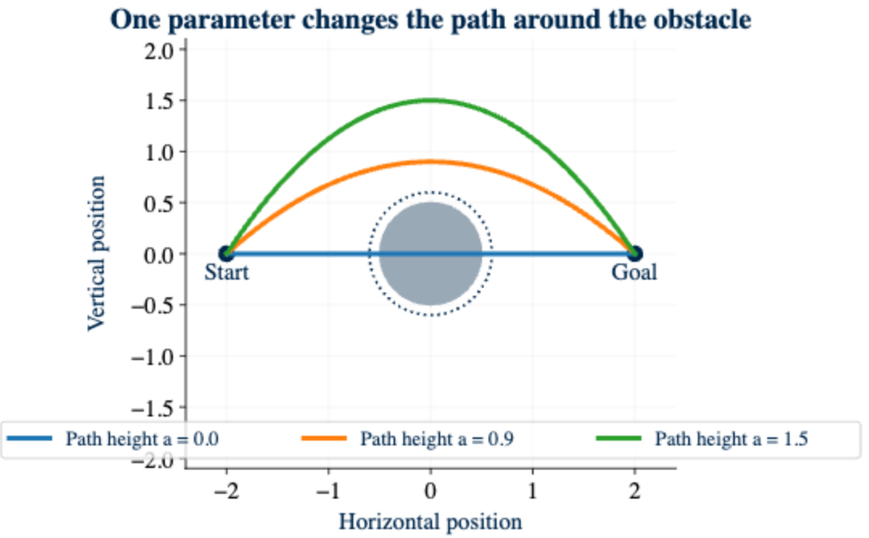
RPCBF motivates the policy-value construction. The common-input inequalities shown here specify our sampled adaptation; the original paper states a separate robust condition.

MBD example: optimize one path-height parameter

Model-based diffusion (MBD) uses model-evaluated costs to refine decision vectors. First consider a scalar decision a: the height of a path between a fixed start and goal.

$$r(s; a) = \begin{bmatrix} -2 + 4s \\ 4a\, s(1 - s) \end{bmatrix}, \qquad s \in [0, 1]$$

Here s indexes progress along the path; it is not physical time. The midpoint has height a.



Minimize path length, subject to clearance from the obstacle. Each sampled value of a produces one path to evaluate.

Teaching example only. The monitoring method will instead optimize a nominal policy parameter vector.

MBD: prefer lower-cost decisions, then smooth the density

Let y denote a decision vector. In the path example, $y = a$ is a scalar path height. The model evaluates its cost $J(y)$.

$$p_0(y) \propto \mathbf{1}_{\mathcal{Y}}(y) \exp\left(-\frac{J(y)}{\lambda}\right).$$

p_0: target probability density over decisions.
$\mathcal{Y}$: feasible decision domain.
$\mathbf{1}_{\mathcal{Y}}(y)$: 1 if $y \in \mathcal{Y}$; 0 otherwise.

$\lambda > 0$: temperature, in cost units.
Smaller λ favors lower costs more strongly.
$\propto$: proportional; density is normalized to integrate to 1.

Example

$J(a)$ is path length. $\mathcal{Y}$ contains heights in $[-2, 2]$ whose paths satisfy the clearance requirement.

Why introduce noise?

The target can have separated high-density regions. Gaussian smoothing produces a sequence of less concentrated densities.

MBD estimates the direction toward higher density using model evaluations, without training a score network.

Pan et al., MBD, Eq. (2). A valid target requires a finite, positive normalizing integral.

MBD noise schedule: what alpha controls

Forward noise defines the densities that MBD will traverse in reverse:

$$y_i = \sqrt{\bar{\alpha}_i}\, y + \sqrt{1 - \bar{\alpha}_i}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \qquad \bar{\alpha}_i = \prod_{j=1}^{i} \alpha_j.$$

y: clean decision; y_i: noisy decision at level i.
$i = 1, \ldots, N$: diffusion level; N: total levels.
p_i: probability density of y_i.

$0 < \alpha_i < 1$: signal-variance retention per forward step.
$\bar{\alpha}_i$: cumulative signal-variance retention.
$\bar{\alpha}_0 = 1$: the clean, zero-noise endpoint.

ϵ: standard Gaussian noise vector.
I: identity matrix of the decision dimension.
j: index in the product defining $\bar{\alpha}_i$.

$\sqrt{\bar{\alpha}_i}$ scales the clean decision.
$\sqrt{1 - \bar{\alpha}_i}$ scales the added noise.
Example: $\bar{\alpha}_i = 0.25$ gives scales 0.5 and $\sqrt{0.75}$.

Forward: $0 \to N$, more noise. MBD optimization: $N \to 0$, less noise. Diffusion level is not physical time.

Pan et al., Eqs. (3)–(4). The controller coefficient is denoted κ here, separately from diffusion α_i.

One MBD iteration: sample, evaluate and weight

At level i, draw K decision vectors around the current noisy vector y_i:

$$y^{[k]} \sim \mathcal{N}\left(\frac{y_i}{\sqrt{\bar{\alpha}_i}}, \frac{1-\bar{\alpha}_i}{\bar{\alpha}_i} I\right), \qquad k = 1, \ldots, K.$$

K: number of sampled decisions per level.
$[k]$: sample index; $y^{[k]}$: one sampled decision.
These samples are in clean decision coordinates.

Proposal mean: $y_i/\sqrt{\bar{\alpha}_i}$.
Proposal covariance: $[(1-\bar{\alpha}_i)/\bar{\alpha}_i]I$.
In the scalar example, $y^{[k]} = a^{[k]}$.

Evaluate each sampled decision with the model. Then normalize its target-density weight:

$$w_k = \frac{p_0(y^{[k]})}{\sum_{j=1}^{K} p_0(y^{[j]})}, \qquad \bar{y} = \sum_{k=1}^{K} w_k y^{[k]}.$$

w_k: nonnegative cost weight, with $\sum_k w_k = 1$.
j: summation index over the same K samples.

$\bar{y}$: cost-weighted decision mean.
Infeasible decisions have zero weight. The mean itself need not be feasible.

Pan et al., Eqs. (8)–(9). K counts decisions; POAM's covariance matrix is K_{ZZ}. If all weights vanish, recovery is required before normalization.

Use the weighted mean to refine the noisy vector

The **score** $s_i(y_i) = \nabla_{y_i} \log p_i(y_i)$ points toward increasing log density. Here ∇_{y_i} means a gradient with respect to the noisy decision vector.

$$\hat{s}_i = \frac{-y_i + \sqrt{\bar{\alpha}_i}\,\bar{y}}{1 - \bar{\alpha}_i}, \qquad y_{i-1} = \frac{y_i + (1 - \bar{\alpha}_i)\hat{s}_i}{\sqrt{\alpha_i}}.$$

$\hat{s}_i$: score estimate from the K model evaluations.
The hat denotes an estimate, not the exact score.
y_{i-1}: updated noisy decision at the next level.

First add the score correction to y_i.
Then divide by $\sqrt{\alpha_i}$ to change levels.
No derivative of the rollout cost is required.

Substituting the score reveals exactly what this update does:

$$y_i + (1 - \bar{\alpha}_i)\hat{s}_i = \sqrt{\bar{\alpha}_i}\,\bar{y} \quad \Longrightarrow \quad \boxed{y_{i-1} = \sqrt{\bar{\alpha}_{i-1}}\,\bar{y}}.$$

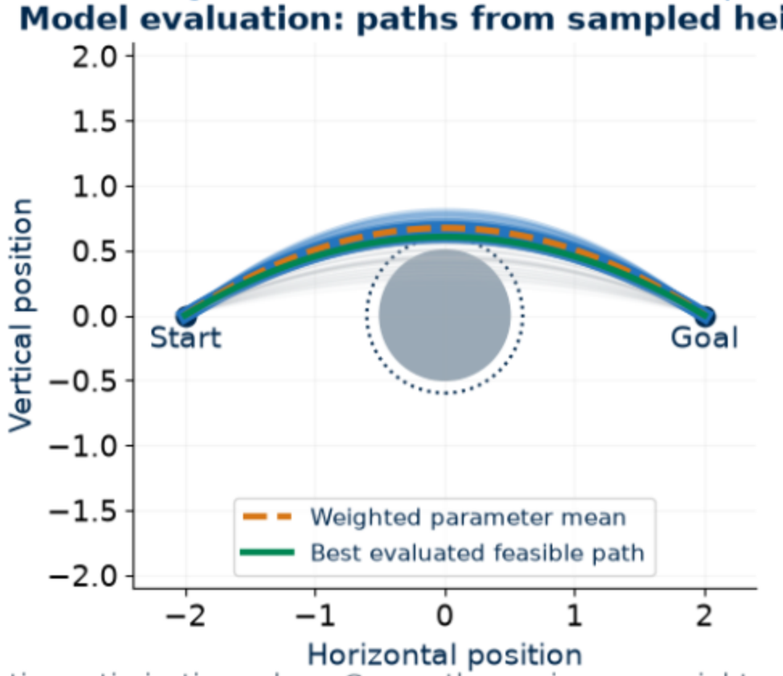
The next sampling mean is $y_{i-1}/\sqrt{\bar{\alpha}_{i-1}} = \bar{y}$. Its covariance decreases to $[(1 - \bar{\alpha}_{i-1})/\bar{\alpha}_{i-1}]I$.

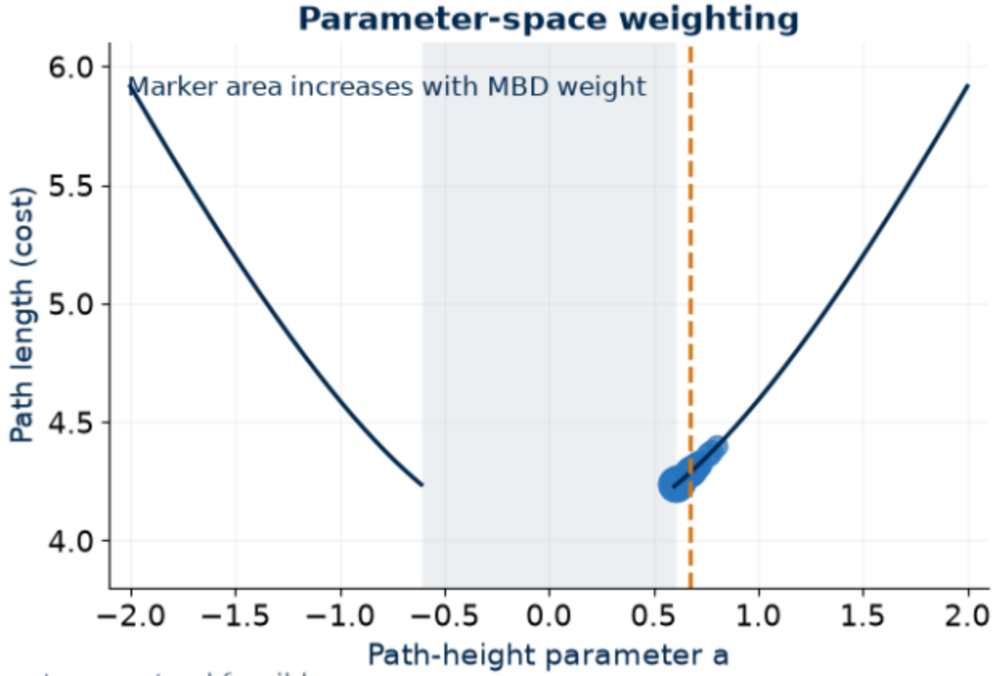
Pan et al., MCSA Eq. (6) and score Eq. (9). At $i = 1$, $y_0 = \bar{y}$ and reverse iterations end. This deterministic update does not guarantee a feasible mean.

Watch MBD refine paths in a simple environment

MBD iteration 8 of 8 | diffusion level i = 1

3. Form the weighted mean; use the score to update the next level





Illustrative optimization only • Grey paths receive zero weight • The weighted mean is not guaranteed feasible

Starts paused. Use Next step or the timeline to move through all eight iterations.

Blue: evaluated paths · Orange dashed: weighted parameter mean · Green: best evaluated feasible path.

One MBD iteration, step by step

8c · Recenter and narrow sampling

Noisy optimizer coordinate y

Start: $y_i = 0.302$

Add 0.244 × 0.553

After score correction: 0.437

Divide by $\sqrt{\alpha_i} = 0.879$

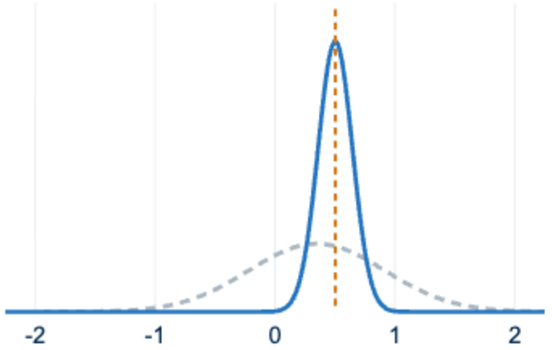
Rescaled: $y_{i-1} = 0.497$

All three rows use the same y scale.

This changes the optimizer, not the robot.

Clean path-height coordinate a

Gaussian density · same vertical scale



Before: mean 0.347 · std. 0.567

Next: mean 0.502 · std. 0.143

View: −2.2 to 2.2; Gaussian tails continue.

1. Read the current noisy vector
2. Sample 64 path-height values
3. Construct each path
4. Evaluate length and clearance
5. Normalize cost weights
6. Compute the weighted mean
7. Estimate the score
8. **Denoise: move → rescale → resample**

8c. The next Gaussian is centered at ā and has smaller standard deviation. Gray dashed is the old proposal; blue is the next proposal.

$$\text{Next mean} = \bar{a}, \qquad \text{next standard deviation} = \sqrt{(1-\bar{\alpha}_{i-1})/\bar{\alpha}_{i-1}}$$

Step 8 has four frames: score correction, rescaling, next sampling distribution, and new paths. Nothing here is physical robot motion.

3 Problem formulation

Predict the monitoring delivered by a trajectory under spatial disturbance.

The decision, its execution, and the cost being minimized

The trajectory decision

At replanning time t_k, the inputs are the attained robot state x_k, clarity vector q_k, disturbance belief $\mathcal{B}_k^d$, and known environment geometry.

Choose $\theta \in \Theta$, a vector that specifies a **nominal reference trajectory** over a prediction horizon T.

The tracking controller requests an input along that reference:

$$u_{\mathrm{nom}}(\tau) = \pi^{\mathrm{nom}}(\tau, x(\tau); \theta, \mathcal{B}_k^d), \qquad 0 \le \tau \le T.$$

k: physical replanning index.
θ: trajectory parameters.
Θ: bounded parameter domain.
τ: elapsed time within a prediction.
π^{nom}: reference tracking law.
u_{nom}: requested input.

The reference representation and vehicle model are specified later, in **Implementation**.

Motion, monitoring, and soft planning constraints

For a possible spatial disturbance field d, predict

$$\dot{x} = f_d(x, u, d(\Psi(x))), \qquad x(0) = x_k, \qquad q(0) = q_k.$$

The clarity model from the preliminaries gives $q(\tau)$ from the predicted sensing positions. The monitoring cost is

$$J(\theta; d) = \int_0^T D(q(\tau))\, d\tau, \qquad D(q) = \frac{1}{N_p} \sum_{j=1}^{N_p} [\bar{q}_j - q_j]_+.$$

State and environment limits enter through a soft penalty:

$$P(\theta; d) = \int_0^T \sum_a \omega_a [h_a(x(\tau))]_+^2 \, d\tau.$$

D: mean positive clarity deficit.
N_p: number of monitored cells.
$[z]_+ = \max(z, 0)$.

$h_a > 0$: violation of constraint a.
$\omega_a \geq 0$: its penalty weight.
Applied inputs remain in actuator set $\mathcal{U}$.

Planning accounts for tracking and constraint effects. It does not require every predicted execution to be safe.

Average performance and optimistic performance

Disturbance-aware planning: minimize average predicted monitoring cost, including average constraint penalties.

Proposed planning: prefer a favorable fraction of predicted monitoring outcomes, while retaining the average constraint penalty over all sampled fields.

$$\mathcal{B}^d(\tau) = \mathcal{B}_k^d \qquad 0 \leq \tau \leq T$$

During a planning call

The disturbance belief stays fixed. The controller responds to each predicted state.

During physical execution

Actual measurements update the belief. The next planning call uses that updated belief.

> **This method uses optimism and real online learning. It does not predict hypothetical belief updates or a value of learning.**

4 Method

The complete algorithm, followed by its components.

All equations in this section are independent of the chosen vehicle and time discretization

Proposed algorithm

Input: attained state and clarity, current disturbance belief, environment, and previous trajectory continuation.

1. Draw spatial disturbance fields from the current belief. Keep them fixed for this planning call.
2. Evaluate initial nominal trajectories and initialize the noisy trajectory parameter vector.
3. At each diffusion level, sample trajectory parameters and predict the resulting controlled motion and sensing in every performance field.
4. Score each trajectory using favorable monitoring outcomes plus average constraint penalties. Retain its evaluated parameters and costs.
5. Use the retained evaluations to form the next sampling distribution. Correct the sample weights and perform the MBD denoising update. Repeat while levels remain.
6. After the last level, reevaluate the final choices in fresh shared fields and select the lowest-scoring evaluated trajectory.
7. Execute for one replanning interval, collect real measurements, update the disturbance belief, and return to step 1 from the attained state and clarity.

Architecture and the two loops

Current state, clarity, and disturbance belief

↓

MBD search: sample trajectory parameters → predict controlled motion and sensing → score → denoise

Repeat only this search while diffusion levels remain

↓ after the last level

Fresh-field comparison and trajectory selection

↓

Physical execution → real measurements → updated state, clarity, and belief

The next physical replanning time returns to the first line

> **The inner search changes the trajectory parameters. Physical execution changes the information available to the next planning call.**

Step 1: spatial fields for prediction and control

The current belief defines a distribution $\mathcal{P}_k^d$ over spatial disturbance fields.

Field set	Definition	Purpose
Performance fields	$d^{[s]} \sim \mathcal{P}_k^d,\ s = 1, \ldots, S$	Predict the monitoring and penalty for one possible execution
Controller construction fields	$\tilde{d}^{[\ell]},\ \ell = 1, \ldots, L$	Evaluate how the controller responds to possible disturbances

Each field is one coherent spatial function. A trajectory queries that same function at its changing position.

S: performance-field count.
L: construction-field count.

$[s]$, $[\ell]$: field indices.
Construction fields use the same rule as the runtime controller.

> **All trajectory decisions are compared using shared fields. A new independent field set is used for final selection.**

Steps 2–3: a nominal trajectory produces several executions

For each sampled parameter vector θ, keep the reference fixed and predict one execution per performance field:

$$
\begin{aligned}
u_{\mathrm{nom}}^{[s]}(\tau) &= \pi^{\mathrm{nom}}(\tau, x^{[s]}(\tau); \theta, \mathcal{B}_k^d), \\
u^{[s]}(\tau) &= \Gamma_k\left(x^{[s]}(\tau), u_{\mathrm{nom}}^{[s]}(\tau)\right), \\
\dot{x}^{[s]} &= f_d\left(x^{[s]}, u^{[s]}, d^{[s]}(\Psi(x^{[s]}))\right).
\end{aligned}
$$

$x^{[s]}$: predicted executed state in field s.
$u^{[s]}$: input applied in that prediction.

Γ_k: runtime input filter reconstructed from the fixed belief and current queried state.

All executions start from x_k, q_k. Their positions, inputs, arrival times, and sensing can differ.

> **The planner scores the motion produced by the controller and disturbance. It does not assume perfect reference tracking.**

Step 3: the controller evaluates a continuation

At a queried state x, the controller tests a specified **design policy** π_D: the continuation used to evaluate constraint consequences, such as braking.

$$\dot{z}^{[\ell]} = f_d\left(z^{[\ell]}, \pi_D(z^{[\ell]}; \mathcal{B}_k^d), \tilde{d}^{[\ell]}(\Psi(z^{[\ell]}))\right), \qquad z^{[\ell]}(0) = x.$$

Let $s_{a\ell}(\sigma; x)$ interpolate constraint values $h_a(z^{[\ell]}(\sigma))$. Define

$$V_{a,k}(x) = \max_{\ell=1,\ldots,L} \max_{0\le\sigma\le T_B} s_{a\ell}(\sigma; x).$$

$z^{[\ell]}$: design-policy trajectory.
T_B: continuation evaluation horizon.
σ: time along that continuation.

$V_{a,k}$: largest tested constraint value.
$s_{a\ell}$: interpolated value for constraint a and field ℓ.

A reference that is easy to track in calm conditions can trigger a different controller response in disturbance.

Policy-value construction inspired by RPCBF. This is a component of the execution prediction, not a planner safety guarantee.

Step 3: the filter changes the requested input when needed

For dynamics affine in the input, write $f_d(x, v, \delta) = f_0(x, \delta) + G(x, \delta)v$. The tested disturbance at x is $\delta_\ell = \tilde{d}^{[\ell]}(\Psi(x))$.

The filter first seeks the closest admissible input satisfying all tested value-rate conditions:

$$\Gamma_k(x, u_{\text{nom}}) \in \arg\min_{v \in \mathcal{U}} \tfrac{1}{2}\|v - u_{\text{nom}}\|_2^2$$
$$\text{subject to } \nabla_x V_{a,k}^{\mathsf{T}}[f_0(x, \delta_\ell) + G(x, \delta_\ell)v] \le -\kappa V_{a,k}, \quad \forall a, \ell.$$

v: trial control; f_0: drift; G: input matrix.
$\nabla_x V_{a,k}$: sensitivity to the queried state.
$\kappa > 0$: value-rate coefficient.
$\mathcal{U}$: hard actuator limits.

If these conditions cannot all be met, the declared fallback permits value-rate slack while preserving the actuator limits.

The planner reproduces that fallback and its motion. It does not reject a trajectory solely because the filter used slack.

Step 3: propagate motion before crediting sensing

For every performance field, repeat the following along the prediction:

1. Query the tracking law at the reached state and reference time.
2. Recompute the controller values and their state derivatives there; obtain the applied input.
3. Propagate the disturbed dynamics with that input.
4. Compute sensing at the reached position, evolve clarity, and accumulate costs.

$$\dot{q}_j^{[s]} = \frac{C_j (x^{[s]})^2}{R}(1 - q_j^{[s]})^2 - Q_j (q_j^{[s]})^2.$$

C_j: sensing quality at cell j.
$R > 0$: measurement-noise level.
$Q_j \geq 0$: monitored-process noise.
$q_j^{[s]}$: clarity in that execution.

A fixed belief does not imply a fixed controller correction or a fixed disturbance vector along the path.

Step 4: monitoring and constraint costs

Each performance field produces a monitoring cost and a penalty:

$$J_s(\theta) = \int_0^T D(q^{[s]}(\tau))\, d\tau, \qquad P_s(\theta) = \int_0^T \sum_a \omega_a [h_a(x^{[s]}(\tau))]_+^2 \, d\tau.$$

Compute their sample means:

$$\overline{J}(\theta) = \frac{1}{S}\sum_{s=1}^{S} J_s(\theta), \qquad \overline{P}(\theta) = \frac{1}{S}\sum_{s=1}^{S} P_s(\theta).$$

J_s: delivered monitoring cost in field s.
P_s: constraint penalty in that field.

$\overline{J}$: average monitoring cost.
$\overline{P}$: average constraint penalty.

Wind and controller intervention affect cost through the resulting motion and sensing. Uncertainty reduction receives no separate reward.

Step 4: favorable monitoring outcomes, all constraint penalties

Sort the monitoring costs: $J_{(1)} \leq \cdots \leq J_{(S)}$. Choose a favorable fraction $\eta \in (0, 1]$.

$$c_s = \min\{1, \max\{\eta S - s + 1, 0\}\}, \qquad J_\eta(\theta) = \frac{1}{\eta S} \sum_{s=1}^{S} c_s J_{(s)}(\theta).$$

The score optimized by MBD is

$$\boxed{F_k(\theta) = J_\eta(\theta) + \overline{P}(\theta).}$$

c_s: weight on sorted outcome s.
$\eta = 1$: average monitoring cost.
$\eta = 0.5$: favorable half of the costs.

F_k: complete planning score.
$\overline{P}$: mean penalty from every field, including unfavorable outcomes.

Optimism applies to monitoring performance. It does not discard unfavorable constraint penalties.

Step 4: an arithmetic example

Illustrative values only. Four performance fields and $\eta = 0.5$. Lower cost is better.

Reference trajectory	Sorted monitoring costs	Mean monitoring	Favorable half	Mean penalty	Score F_k
Established route	$9, 10, 10, 11$	10	9.5	0	9.5
Uncertain alternative	$6, 8, 14, 20$	12	7	1	**8**
Constrained alternative	$6, 8, 14, 20$	12	7	5	12

The uncertain alternative can be chosen despite its worse average monitoring. A large mean constraint penalty can reverse that preference.

> **This example explains the decision criterion. It is not evidence that early exploration improves a later mission.**

Step 5: the target and the ordinary MBD proposal

The clean decision is $y_0 = \theta$. Define the target density

$$p_{0,k}(\theta) = Z_k^{-1} \mathbf{1}_\Theta(\theta)\, \chi_k(\theta) \exp[-F_k(\theta)/\lambda].$$

Z_k: normalizing integral.
$\chi_k = 1$: numerical evaluation is finite.
$\chi_k = 0$: numerical failure.

$\lambda > 0$: cost temperature.
$\mathbf{1}_\Theta$: bounded-domain indicator.
Constraint slack alone does not set $\chi_k = 0$.

At reverse diffusion level i, ordinary MBD samples clean parameters from

$$\phi_i(\theta) = \mathcal{N}\left(\theta; \frac{y_i}{\sqrt{\bar{\alpha}_i}}, \sigma_i^2 I\right), \qquad \sigma_i^2 = \bar{\alpha}_i^{-1} - 1.$$

y_i, α_i, $\bar{\alpha}_i = \prod_{j=1}^{i} \alpha_j$, and identity matrix I are the diffusion quantities defined in the preliminaries. i is a diffusion level, not physical time.

Step 5: which evaluated trajectories receive more samples?

The archive $\mathcal{A}_k$ contains evaluated trajectory parameters and their costs. Its average-cost incumbent is

$$\theta_{\mathrm{inc}} \in \arg\min_{\theta \in \mathcal{A}_k} \{\overline{J}(\theta) + \overline{P}(\theta)\}.$$

For archive entry a, define

$$b_a = \overline{J}(\theta_a) - J_\eta(\theta_a), \qquad a_a = [F_k(\theta_{\mathrm{inc}}) - F_k(\theta_a)]_+ \, \mathbf{1}\{b_a > 0\}.$$

θ_a: parameters of archive entry a.
b_a: difference between average and favorable monitoring cost.

a_a: priority for additional sampling.
Only finite, in-domain evaluations enter this calculation.

Positive priority requires a better optimistic score than the average-cost incumbent and a positive monitoring-cost difference b_a.

This priority is a search rule. It does not estimate future information or the value of learning.

Step 5: draw a new batch and correct its weights

Normalize positive priorities: $\nu_a = a_a / \sum_b a_b$. Form

$$q_i^{\text{prop}}(\theta) = (1-\rho)\phi_i(\theta) + \rho \sum_a \nu_a \mathcal{N}(\theta; \theta_a, \sigma_i^2 I), \qquad 0 \le \rho < 1.$$

Draw K fresh parameter vectors $\theta_n \sim q_i^{\text{prop}}$. Their weights are

$$\widetilde{w}_n = \mathbf{1}_\Theta(\theta_n)\chi_k(\theta_n)e^{-F_k(\theta_n)/\lambda}\frac{\phi_i(\theta_n)}{q_i^{\text{prop}}(\theta_n)}, \qquad w_n = \frac{\widetilde{w}_n}{\sum_{m=1}^K \widetilde{w}_m}.$$

ρ: sampling fraction assigned to archive entries.
ν_a: normalized priority; K: batch size.
n, m: indices within the new batch.

q_i^{prop}: full sampling density.
$\phi_i / q_i^{\text{prop}}$: importance correction.
No positive priorities: use $q_i^{\text{prop}} = \phi_i$.

Use the full mixture density in the denominator. Retained old evaluations are not treated as fresh samples.

Step 5: denoise and decide whether to repeat

Compute the weighted trajectory parameters and apply the MBD update:

$$\bar{\theta} = \sum_{n=1}^{K} w_n \theta_n, \qquad \hat{s}_i = \frac{-y_i + \sqrt{\bar{\alpha}_i}\bar{\theta}}{1 - \bar{\alpha}_i},$$

$$y_{i-1} = \frac{y_i + (1 - \bar{\alpha}_i)\hat{s}_i}{\sqrt{\alpha_i}} = \sqrt{\bar{\alpha}_{i-1}}\bar{\theta}.$$

$\bar{\theta}$: weighted parameter mean.
$\hat{s}_i$: estimated score of the noisy target.

$\bar{\alpha}_0 = 1$: clean endpoint.
The next ordinary proposal is centered at $\bar{\theta}$ with less noise.

More levels remain: draw and evaluate a new batch at the next noise level.
Last level completed: evaluate the final vector and proceed to selection.

A parameter mean need not produce a good trajectory. It must be evaluated. An empty-weight batch triggers the declared recovery using an evaluated trajectory.

Step 6: compare final choices in fresh fields

Compare the leading evaluated parameters, the average-cost incumbent, previous-plan continuation, initial trajectories, and final denoised vector.

For each, recompute the same score in a **fresh shared performance-field set**:

$$\theta_k^\star \in \arg\min_{\theta \in \mathcal{C}_k} \left\{ J_\eta^{\mathrm{val}}(\theta) + \overline{P}^{\mathrm{val}}(\theta) \right\}.$$

$\mathcal{C}_k$: finite, in-domain parameter vectors being compared.
val: costs recomputed in the independent selection fields.

$\theta_k^\star$: selected trajectory parameters.
Use the same controller construction fields and fallback rule.

The final choice is an evaluated trajectory. Fresh-field selection still uses the optimistic criterion.

This comparison reduces reuse of the search fields. It is not an independent mission test or a safety certificate.

Step 7: execute, observe, and replan

Execute the selected reference for a replanning interval $\Delta \leq T$. The runtime controller responds to the attained state and currently available belief.

Actual disturbance measurements form the data set $\mathcal{D}_k^{d,\mathrm{real}}$. Update the belief:

$$\mathcal{B}_{k+1}^{d} = \mathrm{POAMUpdate}(\mathcal{B}_k^{d}, \mathcal{D}_k^{d,\mathrm{real}}).$$

The next call starts from attained x_{k+1} and q_{k+1}, the updated belief, and a shifted continuation of the previous reference.

Δ: physical time between planning calls.
$\mathcal{D}_k^{d,\mathrm{real}}$: measurement locations and values.

x_{k+1}, q_{k+1}: attained state and clarity.
Real measurements may update the runtime controller before the next replan.

Future learning is a consequence of real execution. It is not simulated inside the current planning score.

Proposed method, step by step

Illustrative monitoring costs

Trajectory	Monitoring costs	Penalty	Score
Established route	9, 10, 10, 11	0	**9.5**
Uncertain alternative	6, 8, 14, 20	1	**8**
Poor alternative	11, 13, 16, 20	0	**12**
Known costly route	12, 12, 12, 12	0	**12**

Illustrative arithmetic. Finite, in-domain evaluations.

1. Read attained state, clarity, and posterior
2. Predict controlled motion and sensing
3. **Score monitoring and constraint penalties**
4. Choose where to draw the next batch
5. Correct weights and denoise
6. Compare final choices in fresh fields
7. Execute, observe, and update the posterior

Here η = 0.5 retains the two smallest monitoring costs. Add the average penalty from all four fields. The uncertain alternative has monitoring score 7 plus mean penalty 1, giving total score 8. These are illustrative values.

$$F_k(\theta) = \tfrac{1}{2}\left(J_{(1)}(\theta) + J_{(2)}(\theta)\right) + \overline{P}(\theta)$$

Illustrative arithmetic and process explanation. This is not a simulation result.

Method scope and the long-mission hypothesis

Optimism can encourage an early visit whose average predicted monitoring is worse but whose favorable outcomes are useful. Real observations from that visit can change later decisions.

The benefit requires a persistent spatial disturbance, informative observations, useful generalization, and enough remaining mission time to recover the early monitoring loss.

The score does not predict those later belief changes. It therefore does not guarantee informative exploration or a long-term improvement.

> **The method ends here. The next section compares its algorithm with MBD. All numerical and code choices follow in Implementation.**

5 Comparison with MBD

The MBD update is retained; the execution model, score, and sampling rule are specified for this task.

Algorithm comparison

Model-based diffusion

1. Define a decision vector, model-evaluated cost, and target density.
2. Initialize a noisy decision vector.
3. Draw a Gaussian batch at each diffusion level.
4. Evaluate each decision through the model.
5. Compute weighted decisions, estimate the score, and denoise.
6. Evaluate the final decision for the task.

Current proposed method

1. Use nominal trajectory parameters as the decision.
2. Evaluate controlled motion and sensing in spatial disturbance fields.
3. Score favorable monitoring outcomes plus all mean constraint penalties.
4. Adapt the Gaussian proposal using retained evaluations.
5. Include the proposal-density ratio and apply the same score and denoising equations.
6. Compare final choices in fresh fields; execute and learn from actual measurements.

Pan et al., Model-Based Diffusion for Trajectory Optimization, Algorithm 2 and Eqs. (6), (8)–(9). Optimistic scoring and the corrected archive mixture are additions.

What is optimized, predicted, and learned

Quantity	Role in the current method
Nominal trajectory parameters θ	Optimized by MBD
Controlled states, applied inputs, clarity	Outputs of prediction for each sampled field
Design policy and controller rule	Specified components; not optimized by MBD
Disturbance belief during prediction	Fixed for that planning call
Disturbance belief during execution	Updated from actual measurements
Future monitoring benefit of learning	Experimental hypothesis; not a separate term in the score

The following implementation uses a particular planar reference model and nonlinear quadrotor execution. Those choices are not part of the generic formulation above.

6 Implementation

The exact numerical choices used by the current corrected experiments.

Reference trajectories, quadrotor execution, disturbance inference, search budget, and measured computation

Experiment and clock settings

Quantity	Current corrected two-region configuration
Mission duration	300 s
Prediction horizon T	45 s
Planning time step δt	0.2 s; 225 intervals
Replanning interval Δ	5 s; execute 25 reference intervals
Nonlinear physics step	0.01 s; 100 Hz
Runtime outer-control/filter period	0.05 s; 20 Hz
Actual disturbance-observation period	1 s; runtime belief updates between replans

These are offline simulations. The recorded mission clock does not wait for the measured planning wall time.

The requested 10 s horizon and ≤2 s planning target has not been demonstrated. Runtime tuning was stopped for this review.

The nominal trajectory has 450 independent parameters

Use a planar double-integrator reference with state $z_n^{\mathrm{ref}} = (p_n^{\mathrm{ref}}, v_n^{\mathrm{ref}})$:

$$a_n = a_{\max} \tanh(\theta_n), \qquad p_{n+1}^{\mathrm{ref}} = p_n^{\mathrm{ref}} + \delta t\, v_n^{\mathrm{ref}} + \tfrac{1}{2}\delta t^2 a_n,$$

$$v_{n+1}^{\mathrm{ref}} = v_n^{\mathrm{ref}} + \delta t\, a_n, \qquad n = 0, \dots, 224.$$

$p_n^{\mathrm{ref}}, v_n^{\mathrm{ref}} \in \mathbb{R}^2$: reference position and velocity.
$a_n \in \mathbb{R}^2$: nominal acceleration held over one interval.

$\theta_n \in \mathbb{R}^2$: two independent decision entries.
$a_{\max} = 2.2\ \mathrm{m/s^2}$ per axis.
$\theta = (\theta_0, \dots, \theta_{224}) \in [-12, 12]^{450}$.

Every acceleration interval can change during MBD. There is no temporal basis, fixed route selector, or reduced set of control knots.

quarter/continuous.py: controls, nominal_reference, continuation

Tracking and wind compensation inside prediction

The predicted planar executed state is $z = (p, v)$. The tracking request is

$$u_{\text{nom}} = a_n + k_p(p^{\text{ref}} - p) + k_v(v^{\text{ref}} - v) - a_{\text{drag}}(v, \mu_k(p)).$$

For performance field $d^{[s]}$, the planner propagates

$$\dot{p} = v, \qquad \dot{v} = \Gamma_k(z, u_{\text{nom}}) + a_{\text{drag}}(v, d^{[s]}(p)).$$

$\mu_k(p)$: current posterior wind mean.
a_{drag}: known acceleration from relative air velocity.
$k_p = 4.412\ \text{s}^{-2}$, $k_v = 7.353\ \text{s}^{-1}$.

$p^{\text{ref}}, v^{\text{ref}}, a_n$: reference at that time.
Wind is air velocity in m/s, not an unknown vehicle parameter.

The prediction uses RK4 and recomputes the filter every 0.2 s. The actual quadrotor has nonlinear attitude and motor dynamics.

quarter/continuous.py: rollout · quadrotor/aerodynamics.py: acceleration

Controller values and the braking continuation

The design policy brakes and compensates the posterior mean wind:

$$\pi_D(z) = \mathrm{clip}(-k_B v - a_{\mathrm{drag}}(v, \mu_k(p)),\ -a_{\max}, a_{\max})\,.$$

$k_B = 2\ \mathrm{s}^{-1}$: braking gain.
$a_{\max} = 2.2\ \mathrm{m/s^2}$: per-axis actuator limit.

Two construction fields.
Eight RK4 steps of 0.25 s.
Continuation horizon $T_B = 2$ s.

For each constraint and field, a cubic spline interpolates the nine constraint values, including the starting state. Its maximum includes interior extrema and endpoints.

`jax.jacfwd` differentiates the resulting values with respect to the four queried state entries (p_x, p_y, v_x, v_y).

This short continuation and its derivative are rebuilt inside every filtered prediction step.

safety/flow_policy_value.py: design_policy, values, filter_control · safety/spline.py

The implemented filter and its slack fallback

After computing the values and gradients, assemble inequalities $Au \le b$. The ordinary case projects the tracking request onto those inequalities and $|u_i| \le a_{\max}$.

If infeasible, first compute the smallest common slack:

$$s^\star = \min_{u, s \ge 0} s \quad \text{subject to } Au \le b + s\mathbf{1}, \quad |u_i| \le a_{\max}.$$

Then choose the input closest to the braking policy within the relaxed inequalities.

A: value-gradient coefficients.
b: value-rate bounds including worst tested drift.
$s^\star$: minimum common inequality relaxation.
$\mathbf{1}$: vector of ones.

Two buildings and four boundary constraints give six value inequalities. With four actuator bounds, the 2D projection checks 56 input candidates; the slack LP checks 165 triples of active planes.

safety/qp.py: project, solve, minimum_slack. The fallback preserves actuator limits; it is not declared safe.

Numerical penalties and contact handling

The per-step soft penalty uses center clearance $c(z)$ and speed:

$$\ell_P(z) = [0.6 - c(z)]_+^2 + 2[\|v\|_2 - 3.1]_+^2.$$

$c(z)$: signed distance to the closest building or boundary, in m.

0.6 m: preferred planning clearance.

3.1 m/s: speed threshold.

Both terms are integrated in time and averaged over fields.

The filter uses a 0.55 m center margin. The physical quadrotor radius is 0.275 m.

A predicted physical contact stops the predicted motion and subsequent sensing. Clarity then deteriorates for the remaining horizon; no monitoring after contact is credited.

Filter slack and soft-margin violations do not automatically reject a finite trajectory. Numerical failures and parameters outside the domain receive zero weight.

quarter/continuous.py: segment_contact, rollout, aggregate_costs

Disturbance belief and field generation

Component	Current choice
Sparse posterior	36 inducing positions; relocate after actual observations
Kernel	Attentive kernel; fixed parameters in these experiments
Base correlation lengths	1.75 m and 4.55 m
Initial wind standard deviation	1.2 m/s per component
Observation noise	0.25 m/s per component
Continuous field sampling	16 spectral features plus a 25×25 prior-support grid
Field lookup	80×91 spatial grid; bicubic interpolation; two wind components

The support grid corrects sampled prior covariance. It is not an extra set of observations or a change to posterior inference.

quarter/environment.py: initial_belief, table_sampler · wind/poam_belief.py

RRT provides initial trajectories, not enforced paths

RRT-Connect uses only known geometry and the attained state. Geometric diversity checks retain routes around different sides of obstacles.

For two inspection regions, the configured maximum gives

$$2 \text{ goal regions} \times 3 \text{ path choices} \times 3 \text{ visit counts} \times 2 \text{ speeds} = 36 \text{ initial trajectories}.$$

Add zero acceleration, braking, and the shifted previous trajectory: **39 initial parameter vectors** are scored.

Each RRT path is time-parameterized using quintic segments, with zero velocity at geometric corners. Cruise settings are 1.8 and 2.8 m/s.

> **MBD can change all 450 parameters. The current weights use model cost only; the optional demonstration-likelihood term is off.**

Stops at RRT corners are initialization choices and can bias a small local search. They are not constraints on the optimized trajectory.

quarter/rrt.py: diverse_connections, parameterize, demonstrations · continuous.py: optimizer

Diffusion settings in the recorded experiments

Setting	Value
Reverse levels and batch size	6 levels; 16 new parameter vectors per level
Performance and selection fields	4 during search; 8 fresh fields for selection
Optimistic fraction and temperature	$\eta = 0.5$; $\lambda = 0.3$
Archive proposal	Up to 8 entries; mixture fraction $\rho = 0.35$
Cosine schedule	Offset 0.008; finite schedule interval $[0, 0.05]$
Parallel layout	One search chain; trajectory batch size 16

The largest proposal standard deviation is **0.090 per parameter**. The initial noisy vector is formed around the best evaluated initialization.

> **This is a small local search around evaluated trajectories. It is not evidence of convergence or exhaustive passage discovery.**

quarter/diffusion.py: cosine_schedule · quarter/mbd_core.py: search

How one planning call becomes 912 trajectory rollouts

Stage	Parameter vectors	Fields per vector	Executed-motion predictions
Initial comparisons	39	4	156
Six diffusion batches	$6 \times 16 = 96$	4	384
Final denoised vector	1	4	4
Fresh-field selection	46 entries	8	368
Total	**182 evaluations**		**912**

Selection contains seven search outputs plus all 39 initial entries. Some entries can duplicate the same vector; they are still evaluated again.

$$912 \times 225 = 205{,}200 \text{ forecast filter calls per planning call.}$$

Each filter call includes two eight-step design-policy continuations and state derivatives: **3,283,200 continuation steps before counting derivative work**.

Counts follow the actual array shapes in mbd_core.search and continuous.optimizer. Construction fields are nested inside these predictions.

Nonlinear execution and monitoring

The selected planar reference is flown at 3.2 m altitude using the full rigid-body quadrotor and motor model.

- Differential-flatness mapping supplies attitude and thrust references; geometric attitude control produces motor commands.
- RK4 advances the 22-state model at 100 Hz. Rotation is projected back to a valid rotation matrix after each step.
- The outer tracking/filter command is recomputed at 20 Hz using the actual state. Every second, a measured wind residual updates POAM without restarting the reference clock.
- A finite nadir sensing footprint, Gaussian quality, and building occlusion determine clarity at 84 monitored cells.

The 45° footprint has radius 3.2 m at the nominal altitude. The sensing quality scale is 2.2 m; target clarity is 0.75 and monitored-process noise is 0.014.

quarter/execution.py: executor · quarter/sensing.py · scripts/run_full_control_quarter.py: monitoring and online execution loop

Baselines and the shared execution controller

Method	During planning	During physical execution
Proposed	Sampled spatial disturbance, predicted filter response, optimistic monitoring plus mean penalties	Learned wind compensation and runtime filter
Disturbance-blind planning	Zero disturbance and no predicted filter response	Learned wind compensation; no added constraint filter
Disturbance-blind planning with filter	Same nominal planning as the blind baseline	Same learned compensation and runtime filter as proposed

The vehicle, actuator limits, task, sensing, estimator, and reference tracking gains are shared.

> **Proposed versus filtered blind planning isolates the effect of planning with the disturbance and controller response.**

An average-cost disturbance-aware ablation uses the same disturbed execution model with $\eta = 1$ and an ordinary Gaussian proposal. A collision is a failure to report, not a performance advantage to claim.

Where JAX parallelism is used

Operation	Current implementation
Complete optimizer	`jax.jit` compiles search and final selection
Trajectory evaluations	`lax.map`, batch size 16
Performance fields and construction fields	Nested `vmap`
Prediction over 225 time intervals	`lax.scan`
Eight-step controller continuation	A second `lax.scan`, inside each prediction step
Constraint-value derivatives	`jax.jacfwd` with respect to four state entries
Diffusion levels	`lax.scan`; each level depends on the preceding update

Vectorization reduces dispatch overhead. It does not remove the nested prediction and derivative work.

Current backend: JAX 0.4.38, CPU on macOS arm64. MBD needs no task-cost gradient, but its controller model still computes state derivatives.

7 Compute-time investigation

Measured wall time, work counts, and the expensive nested calculation.

Current 45 s configuration; no runtime parameter tuning in this revision

What was measured

Use the recorded forward-wind state and learned posterior at **mission time 100 s**, with the unchanged configuration above.

- Time the complete optimizer directly and synchronize its JAX outputs before stopping the clock.
- Time the complete planning path: posterior field generation, RRT, diffusion, and fresh-field selection.
- Compile and time search and selection separately for diagnosis. Verify that their selected parameters and score match the complete optimizer.
- Time nested controller and rollout components with dynamic inputs; do not compile a constant-answer benchmark.

Cold calls, warm calls, and nested measurements are reported separately.

Three warm repetitions for large stages; more repetitions for small components. One recorded state, one CPU machine. These measurements do not establish a mission-wide worst-case deadline.

scripts/profile_current_quarter.py · docs/compute_review/current45_dynamic.json

Complete planning time

Timed operation	Warm median	Warm maximum
Complete planning call	**10.75 s**	**10.79 s**
Compiled optimizer: search plus selection	9.30 s	9.33 s
RRT initialization generation	1.12 s	1.12 s
One posterior field-bank generation	32.07 ms	47.94 ms

The current runner generates two 14-field banks per planning call. Fourteen of those 28 fields are used: two construction fields, four search fields, and eight selection fields.

> **The optimizer dominates. Field generation is much smaller; removing it alone cannot deliver a 2 s planning call.**

Complete-call times are measured directly. The other rows are separate synchronized measurements; their medians are not an exact additive decomposition.

Search and final selection both cost seconds

Separately compiled stage	Warm median	Work represented
Training search, including initial evaluations	**5.71 s**	136 trajectory vectors × 4 fields
Fresh-field selection	**3.64 s**	46 entries × 8 fields
Initial evaluations alone	1.65 s	39 vectors × 4 fields
One diffusion batch alone	0.69 s	16 vectors × 4 fields

The last two rows are **parts of the search**, not additional costs to add to the first row. Six batches repeat the same expensive prediction pattern.

> **Fresh-field selection is substantial because it reevaluates all initial entries, including duplicates, under twice as many fields.**

Separate compilation changes fusion and scheduling. The split result matches the complete optimizer, but its component times are diagnostic rather than an exact partition.

The controller prediction is the main added rollout cost

For the same optimized trajectory and the same four physical fields:

Diagnostic evaluation	Warm median
Full 45 s motion and clarity prediction with filter	**57.09 ms**
Same prediction with the filter disabled	10.60 ms
Ratio	**5.4×**

At one queried state, separate nested measurements give:

Nested component	Warm median
Constraint values from design-policy continuations	44.8 µs
Constraint values and state derivatives	112.0 µs
Complete filter call, including projection/fallback logic	124.7 µs

Disabling the filter changes the trajectory and is a diagnostic ablation, not a valid substitute for the method. Nested times overlap and are not added together.

Other measured costs and what the old timer omitted

Operation	Warm median	Frequency
One actual POAM update	0.24 ms	Every 1 s of simulated flight
One second of nonlinear quadrotor execution	2.31 ms	100 physics steps
First optimizer invocation	25.19 s	Includes compilation and the first execution

Existing mission logs record optimizer time in `plans[].seconds`. Their `total_planning_seconds` adds RRT, but excludes field construction and report generation.

Neither log timer includes execution, belief-map plotting, raw NPZ audit compression, or JSON serialization. The new complete-call measurement includes field construction and RRT, and excludes those reporting tasks.

> **Compilation is a startup cost. The roughly 10.75 s warm call remains too slow for a 5 s replanning deadline.**

Current compute diagnosis

1. **Long repeated predictions:** 912 trajectory-field predictions, each with 225 time intervals.
2. **Controller reconstruction inside each interval:** braking continuations, spline maxima, and state derivatives occur about 205,200 times per call.
3. **Large final comparison:** 46 entries are reevaluated in eight fields; duplicate entries are not removed.
4. **Additional RRT work:** geometry search and time parameterization add approximately 1.12 s.

The nominal decision dimension is 450. The immediate expense comes from repeatedly evaluating those decisions through the controller and sensing model.

> **A 10 s horizon may reduce prediction work. No ≤2 s result is claimed; that experiment was stopped for review.**

Configuration fields that do not affect the current path

Configuration entry	Effect in this configuration
`slack_weight = 50`	Unused by the hard projection/minimum-slack fallback; this is not a weighted soft QP
`safety_margin = 0.4`	Replaced by `execution_margin = 0.55` in the hard prediction filter
`guidance_sigma = 1`	Inactive because `guidance_mode = off`
`jerk_limit = 12`	No jerk cost because `jerk_penalty = 0`

The numerical filter is evaluated every 0.2 s in prediction and every 0.05 s in execution. Matching its equations does not make these discretizations identical.

These are current code facts, not proposed algorithm components. They are listed so parameter changes can be interpreted correctly.

Files for independent investigation

Topic	Repository file and entry point
Experiment configuration and complete call	`scripts/run_full_control_quarter.py`: `run`
Prediction, penalties, and final selection	`src/voi_diffusion_ipp/quarter/continuous.py`
MBD proposal, archive, and reverse update	`src/voi_diffusion_ipp/quarter/mbd_core.py`: `search`
Controller values and derivatives	`src/voi_diffusion_ipp/safety/flow_policy_value.py`
Projection and slack fallback	`src/voi_diffusion_ipp/safety/qp.py`
Initialization and nonlinear execution	`quarter/rrt.py`; `quarter/execution.py`
Measured timings and exact configuration	`docs/compute_review/current45_dynamic.json`
Profiling script and reproduction notes	`scripts/profile_current_quarter.py`; `docs/compute_review/README.md`

The following final archive preserves the earlier anticipated-learning formulation and its historical results. It is not the current method.

A

Archived method

Previous anticipated-learning and value-of-learning formulation.

Original equations, interactive explanations, A/B results and implementation notes retained for reference

3 Archived problem formulation

Two planning problems with the same monitoring objective.

Define the decision variable → hold the wind belief fixed → anticipate observations

Archived method · prior formulation and results

Define the decision and the planning horizon

At time t, the planner knows the robot state x_t, clarity vector q_t, wind belief $\mathcal{B}_t^d$, environment geometry and target clarity $\bar{q}$.

Decision: choose parameters $\theta \in \Theta$ of a nominal control policy:

$$u_{\text{nom}}(\tau) = \pi_\theta\big(x(\tau), \mathcal{B}^d(\tau), \tau\big), \qquad 0 \le \tau \le T.$$

Θ: policy parameter domain.
T: planning horizon.
τ: elapsed time from the start of this prediction.

u_{nom}: input requested by the task policy.
u: input applied by the execution controller.
$x(0) = x_t$, $q(0) = q_t$.

The decision is a nominal policy. Its predicted motion and sensing determine the monitoring cost.

Archived method · prior formulation and results

Shared motion constraints and execution controller

Both problems require admissible policy parameters, actuator inputs and predicted states:

$$\theta \in \Theta, \qquad u(\tau) \in \mathcal{U}, \qquad h_a(x(\tau)) \leq 0 \quad \text{for every constraint } a.$$

h_a encodes an environment or state constraint. For example, outside a circular exclusion region, $h_a(x) = R_a^2 - \|\Psi(x) - c_a\|^2$, with center c_a and radius R_a.

The controller from the preliminaries determines the applied input:

$$u = \arg\min_{v \in \mathcal{U}} \frac{1}{2}\|v - u_{\text{nom}}\|^2 \quad \text{s.t.} \quad \nabla_x V_a^\mathsf{T} f_d(x, v, d) + \kappa V_a \leq 0.$$

The value V_a and tested wind values d depend on the available wind belief. The inequality is imposed for each constraint and tested wind value. Here v is the trial input; u is the selected input.

> **The planner evaluates the motion produced by this controller. It cannot choose the nominal route and assume that route is executed.**

The numerical method uses finite field samples and time checks. If the controller's inequalities are infeasible, its prescribed relaxation must be included in prediction. These tests do not establish safety for every possible wind field.

Archived method · prior formulation and results

Problem 1: wind-aware planning with a fixed predicted belief

Choose a nominal policy that minimizes the monitoring deficit of its predicted execution under uncertain wind.

$$\min_{\theta \in \Theta} \; \mathbb{E}_{d \mid \mathcal{B}_t^d} \left[\int_0^T q_d(q(\tau))\, d\tau \right]$$

subject to the dynamics, initial conditions and admissibility requirements:

$$\begin{aligned} \dot{x} &= f_d(x, u, d(\Psi(x))), && x(0) = x_t, \\ \dot{q}_j &= \frac{C_j(x)^2}{R}(1 - q_j)^2 - Q_j q_j^2, && q(0) = q_t, \\ u &\in \mathcal{U}, && h_a(x) \le 0 \quad \forall a. \end{aligned}$$

The controller on the preceding slide chooses u from the request $u_{\text{nom}} = \pi_\theta(x, \mathcal{B}_t^d, \tau)$.

$$\boxed{\mathcal{B}^d(\tau) = \mathcal{B}_t^d \quad \text{for } 0 \le \tau \le T}$$

A possible physical field d is drawn from the current belief. Requirements apply to predicted executions; numerical enforcement checks the sampled fields and times. Real measurements still update the belief between planning calls.

Archived method · prior formulation and results

Problem 2: wind-aware planning with anticipated learning

Choose the policy while predicting how future observations change later inputs, motion and monitoring.

$$\min_{\theta\in\Theta}\ \mathbb{E}_{d,\varepsilon^d|\mathcal{B}_t^d}\left[\int_0^T q_d(q(\tau))\,d\tau\right]$$

Retain all motion and sensing constraints from Problem 1:

$$\dot{x} = f_d(x,u,d(\Psi(x))),\quad \dot{q}_j = \frac{C_j(x)^2}{R}(1-q_j)^2 - Q_j q_j^2,$$

$$x(0)=x_t,\quad q(0)=q_t,\quad u\in\mathcal{U},\quad h_a(x)\le 0\ \ \forall a.$$

The same controller chooses u, now from $u_{\text{nom}} = \pi_\theta(x,\mathcal{B}^d,\tau)$. At a sensing time, $r_m = \Psi(x)$ and

$$y_m^d = d(r_m) + \varepsilon_m^d,\qquad \boxed{\mathcal{B}^{d,+} = \text{POAMUpdate}(\mathcal{B}^d; r_m, y_m^d)}.$$

> **Continue from the reached x, q using the updated belief. The objective and admissibility requirements are unchanged.**

$\varepsilon_m^d \sim \mathcal{N}(0,R_d)$ is measurement noise. The belief starts at $\mathcal{B}_t^d$ and is retained between updates. The expectation includes fields and observation noise; there is no separate learning reward.

Archived method · prior formulation and results

Method: from policy parameters to delivered monitoring

PLANNING · evaluate motion and monitoring over horizon T

1 Current inputs

Robot state

Clarity vector

Wind belief

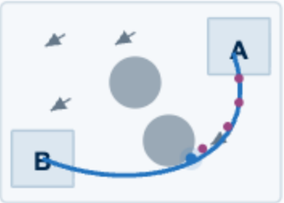
Geometry and targets

2 Sample parameters

θ θ θ

Each vector defines one nominal control policy.

3 Predict execution



Applied inputs

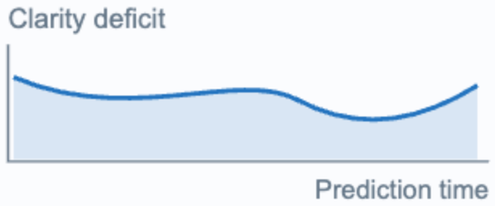
Robot motion

Sensing and clarity

Current belief → wind-aware evaluation

Updated belief → anticipated-learning evaluation

4 Evaluate monitoring



Integrate clarity deficit;
average over possible physical fields.

5 MBD score update

Cost weights → weighted mean → refine

More diffusion levels?

Choose exactly one continuation below.

YES
Repeat MBD

NO · optimization finished

PHYSICAL EXECUTION

6 Execute the next interval

Select and check an evaluated policy.
Apply its inputs; collect measurements.

7 Update measured inputs

Update state, clarity and wind belief.
Start the next planning call.

REAL MEASUREMENTS → NEXT PLANNING CALL

Inside the motion sketch

- Predicted executed motion
- Predicted observation location

The sketch illustrates the prediction, not an additional experiment.

Archived method · prior formulation and results

First distinguish the requested input from the applied input

For a given parameter vector θ, the nominal policy requests

$$u_{\text{nom}} = \pi_\theta(x, \mathcal{B}^d, \tau).$$

The execution controller chooses $u \in \mathcal{U}$ using that request, the current state and the wind belief. The robot then evolves according to

$$\dot{x} = f_d(x, u, d(\Psi(x))).$$

Example

The task policy requests motion toward a monitored region. Near an obstacle, wind and current velocity may require braking or turning.

Planning consequence

The robot may arrive later or sense different cells. Predicting only the requested motion misses that cost.

> **Next we construct the execution controller's wind-dependent constraint test, then use its chosen input in the motion prediction.**

Here d is a possible physical wind field. The controller receives the belief, not privileged knowledge of that field.

Archived method · prior formulation and results

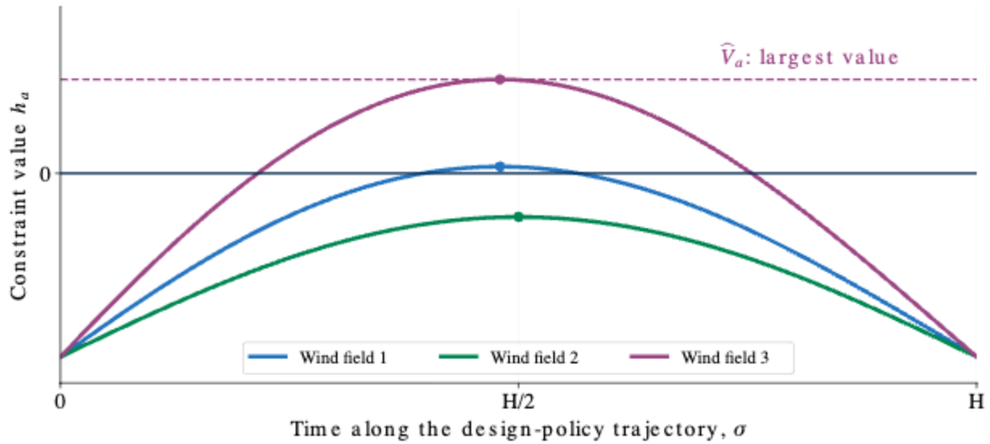
Predict future constraint values under a fixed policy

Use the constraint functions h_a and a fixed design policy π_b from the preliminaries. MBD optimizes θ, the parameters of the nominal task policy π_θ; π_b is fixed.

Draw L wind fields $\tilde{d}^{[\ell]}$ from the belief. Starting at x, follow π_b under each field. Denote the resulting state trajectory by $\phi_b^{[\ell]}(\sigma)$.

Take the largest constraint value over time, then over fields:

$$\widehat{V}_a(x;\mathcal{B}^d) = \max_{\ell=1,\ldots,L} \max_{0\le\sigma\le H} h_a\left(\phi_b^{[\ell]}(\sigma)\right).$$



H: design-policy evaluation horizon.
σ: time within that prediction.
ℓ: wind-field index.

The belief changes the tested trajectories and their largest constraint value.

Illustrative traces. Finite sampled policy-value construction inspired by Knoedler et al.; spline maxima approximate the time maximum.

Archived method · prior formulation and results

Predict which input the execution controller will apply

At the predicted state, choose the input closest to the nominal request:

$$u = \arg\min_{v \in \mathcal{U}} \frac{1}{2}\|v - u_{\text{nom}}\|^2$$

subject to, for every constraint a,

$$\max_{\ell=1,\ldots,L} \nabla_x \widehat{V}_a^{\mathsf{T}} f_d(x, v, \tilde{d}^{[\ell]}(\Psi(x))) + \kappa \widehat{V}_a \leq 0.$$

v is the trial control input in this optimization; u is its selected value. The belief is held fixed when taking the state gradient $\nabla_x \widehat{V}_a$.

If the nominal input satisfies every inequality: $u = u_{\text{nom}}$.
Otherwise: the controller changes the input to meet the tested conditions, when feasible.

> **Wind affects both the value $\widehat{V}_a$ and the predicted rate of change. It can therefore change the applied input even for the same nominal request.**

Control-affine dynamics give a quadratic program. If the inequalities are infeasible, the execution model must reproduce the controller's constraint relaxation; the planner does not assert safety.

Archived method · prior formulation and results

Propagate the applied input, then compute sensing

The chosen input u and the possible physical wind field d determine the next robot state:

$$\dot{x} = f_d(x, u, d(\Psi(x))), \qquad x(0) = x_t.$$

That state determines sensing quality and the clarity evolution:

$$\dot{q}_j = \frac{C_j(x)^2}{R}(1 - q_j)^2 - Q_j q_j^2, \qquad q(0) = q_t.$$

At each prediction step: recompute the nominal request and the execution controller's response at the reached state.

Accumulate monitoring cost: integrate the mean clarity deficit $q_d(q)$ along this predicted motion.

> **The constraint value is part of the execution model. The objective is monitoring quality, not the size of the controller's intervention.**

Archived method · prior formulation and results

Wind-aware evaluation: average costs across possible fields

One possible physical field gives one predicted trajectory and one monitoring cost. Use S coherent fields $d^{[1]}, \ldots, d^{[S]}$ drawn from the current belief.

$$J_s(\theta) = \int_0^T q_d(q^{[s]}(\tau))\, d\tau, \qquad \hat{J}^{\text{fixed}}(\theta) = \frac{1}{S} \sum_{s=1}^{S} J_s(\theta).$$

s indexes a possible physical field.
$q^{[s]}$ is the clarity predicted in that field.
Every evaluation starts from the same x_t, q_t.

The physical field stays fixed along its trajectory.
The controller uses the current belief throughout: $\mathcal{B}^d(\tau) = \mathcal{B}_t^d$.

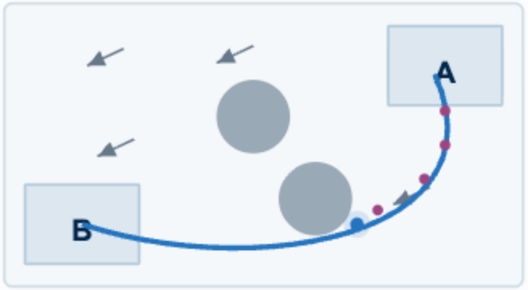
A short nominal route can have a higher cost if wind and controller intervention reduce its delivered sensing.

The S physical-field predictions evaluate monitoring. The L short design-policy predictions on the previous slides construct the controller's constraint test; these have different roles.

Archived method · prior formulation and results

Anticipated learning: update, then continue

1 Predict until an update



Use the current wind belief.

Predict motion and observations.

At the update time

Reached robot state

Reached clarity vector

Predicted wind observations

Same state and clarity;

different belief for later inputs.

Wind-aware continuation

Retain the current wind belief.

Continue from the reached state and clarity.

Anticipated-learning continuation

Update POAM from predicted observations.

Recompute the policy and controller response.

Continue from the reached state and clarity.

A predicted observation changes only the continuation after it becomes available. The robot state and clarity are carried forward.

Archived method · prior formulation and results

Predict one observation and its belief update

At a scheduled sensing time, the predicted robot reaches position r_m. In the chosen possible physical field d, generate

$$y_m^d = d(r_m) + \varepsilon_m^d, \qquad \varepsilon_m^d \sim \mathcal{N}(0, R_d).$$

Use that location and measurement in the same POAM update used during execution:

$$\mathcal{B}^{d,+} = \mathrm{POAMUpdate}\left(\mathcal{B}^d;\, r_m, y_m^d\right).$$

m: observation index.
y_m^d: predicted noisy wind observation.
R_d: measurement-noise covariance.

$\mathcal{B}^d$: belief before the observation.
$\mathcal{B}^{d,+}$: belief after the observation.
A batch is handled using all locations observed since the last update.

> **Example: an observation can reveal stronger crosswind than expected. Later predicted inputs then respond to that updated estimate.**

The observation is generated from the possible field used for this trajectory. The controller learns about it only through the predicted measurement.

Archived method · prior formulation and results

What changes after a predicted observation?

From the reached state x, recompute the nominal request with the updated belief:

$$u_{\mathrm{nom}} = \pi_\theta(x, \mathcal{B}^{d,+}, \tau).$$

Nominal policy: wind compensation and other belief-dependent control decisions respond to $\mathcal{B}^{d,+}$.

Execution controller: recompute the design-policy value and input inequalities from $\mathcal{B}^{d,+}$.

Continuation: propagate the selected input under the same physical field, evolve clarity and predict later observations.

> **Keep the same policy parameters θ. The policy responds to new information; there is no nested MBD solve at each predicted update.**

The anticipated-learning cost includes the initial motion, sensing and later continuation. The following slides express its benefit through matched predictions.

Archived method · prior formulation and results

Two matched predictions define the value of learning

Fix one policy parameter vector θ. Each draw s contains a physical field and an observation-noise stream. Both predictions use that field and the same initial state and clarity.

$$J_s^{\text{fixed}} = \int_0^T q_d(q_s^{\text{fixed}}(\tau))\, d\tau, \qquad J_s^{\text{learn}} = \int_0^T q_d(q_s^{\text{learn}}(\tau))\, d\tau.$$

Fixed belief: retain $\mathcal{B}_t^d$ throughout the prediction.

Anticipated learning: assimilate observations when available and continue from the reached state and clarity.

$$\Delta_s = J_s^{\text{fixed}} - J_s^{\text{learn}}.$$

$\Delta_s > 0$: learning reduces monitoring cost.

$\Delta_s < 0$: learning increases monitoring cost.

Both costs cover the same full horizon, including the motion used to obtain information.

Archived method · prior formulation and results

How often and how much does learning help?

Among the S matched predictions, let n_+ count those with $\Delta_s > 0$.

$$\hat{p} = \frac{n_+}{S}, \qquad \hat{g} = \frac{1}{n_+} \sum_{s:\, \Delta_s > 0} \Delta_s.$$

$\hat{p}$: estimated probability that learning helps.

$\hat{g}$: mean cost reduction among the predictions where learning helps. Set it to zero if $n_+ = 0$.

$$\widehat{V}^+ = \hat{p}\,\hat{g} = \frac{1}{S} \sum_{s=1}^{S} \max(\Delta_s, 0).$$

Probability times conditional improvement gives the expected positive benefit.

These quantities describe one policy. They do not yet account for outcomes where its learning-dependent execution performs worse.

Archived method · prior formulation and results

Signed learning value includes deterioration

The expected deterioration is

$$\widehat{D} = \frac{1}{S}\sum_{s=1}^{S} \max(-\Delta_s, 0).$$

Subtract it from the positive benefit:

$$\boxed{\widehat{V}^{\text{net}} = \hat{p}\,\hat{g} - \widehat{D} = \frac{1}{S}\sum_{s=1}^{S} \Delta_s}.$$

Illustrative cost differences Δ_s	$+2$	$+1$	-1	0
Interpretation	Helps	Helps	Harms	No change

Here $\hat{p} = 0.5$, $\hat{g} = 1.5$, $\widehat{D} = 0.25$, so $\widehat{V}^{\text{net}} = 0.50$.

A policy receives credit for beneficial outcomes and a penalty for deterioration.

Archived method · prior formulation and results

Complete task cost with anticipated learning

Average the two matched full-horizon costs:

$$\hat{J}^{\text{fixed}} = \frac{1}{S}\sum_s J_s^{\text{fixed}}, \qquad \hat{J}^{\text{learn}} = \frac{1}{S}\sum_s J_s^{\text{learn}}.$$

$$\boxed{\hat{J}^{\text{fixed}} - \widehat{V}^{\text{net}} = \hat{J}^{\text{learn}}}.$$

MBD minimizes $\hat{J}^{\text{learn}}$. This includes the information-gathering motion and its later payoff.

Illustrative policy	$\hat{J}^{\text{fixed}}$	$\widehat{V}^{\text{net}}$	$\hat{J}^{\text{learn}}$
Direct route	10	1	**9**
Detour to learn	14	4	10

Choose the direct route here. The detour has a larger learning value but a higher complete task cost.

Do not subtract learning value again from $\hat{J}^{\text{learn}}$. That would count the benefit twice.

Archived method · prior formulation and results

MBD now optimizes the complete monitoring prediction

The MBD decision is the nominal policy parameter vector: $y = \theta$. A **sampled policy parameter vector** $\theta^{[k]}$ defines one policy whose cost is evaluated by the preceding prediction.

$$w_k \propto \mathbf{1}_{\Theta_{\text{eval}}}(\theta^{[k]}) \exp\left(-\frac{\hat{J}(\theta^{[k]})}{\lambda}\right).$$

Here $\Theta_{\text{eval}} \subseteq \Theta$ contains evaluable policy parameters satisfying the specified numerical feasibility checks. Normalize the weights to sum to one.

$$\bar{\theta} = \sum_k w_k \theta^{[k]}.$$

Wind-aware: $\hat{J} = \hat{J}^{\text{fixed}}$ uses the fixed-belief execution prediction.

Anticipated learning: $\hat{J} = \hat{J}^{\text{learn}}$ includes chronological belief updates.

> **Use $\bar{\theta}$ in the MBD score update from the preliminaries. Average costs across physical fields before exponentiating.**

Weights act on nominal parameters. Executed trajectories supply the costs; they do not replace parameters in the weighted mean.

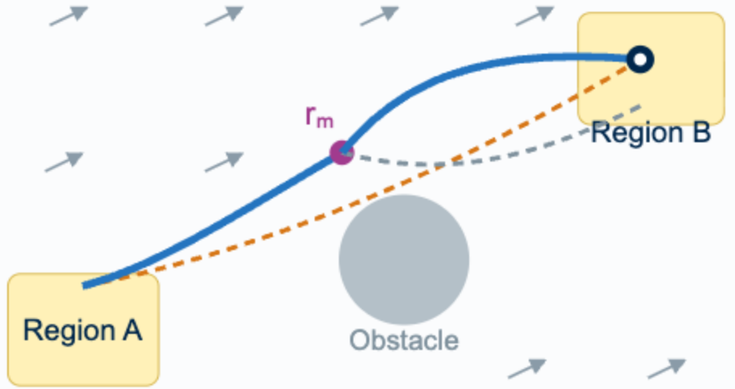
Archived method · prior formulation and results

Follow one policy's predicted execution

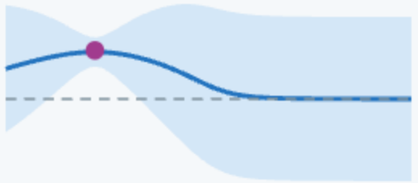
Anticipated learning: continue from the reached state

Anticipated learning changes the continuation

Predicted motion in one possible field



Wind belief before / after



Blue: mean ±2 std. dev.

Recompute policy and controller.
No nested MBD optimization.

Gray dashed: continuation with the frozen belief

1. Begin at the measured state
2. Fix one sampled policy vector
3. Predict the applied input
4. Advance motion and clarity
5. Generate a predicted observation
6. Update the predicted wind belief
7. **Continue with the same parameters**
8. Complete the monitoring prediction

Keep θ fixed. The updated belief changes the nominal request and execution controller. The frozen-belief continuation is shown only for comparison.

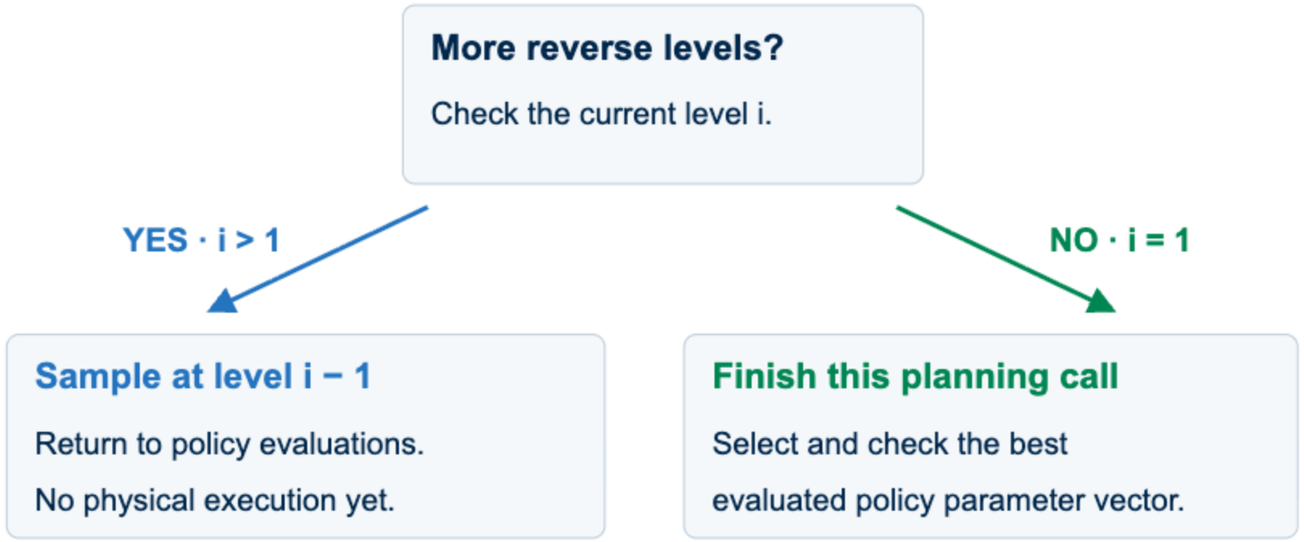
$$u_{\text{nom}} = \pi_\theta(x, \mathcal{B}^{d,+}, \tau) \quad \text{with the same } \theta$$

Schematic explanation of the algorithm, not a new mission result. Orange: nominal motion · Blue: predicted execution · Purple: predicted observation.

Archived method · prior formulation and results

Turn predicted monitoring costs into the next action

End of reverse level: choose the appropriate branch



These are exclusive branches, not two actions to perform in sequence.

1. Evaluate every sampled policy vector
2. Average across possible wind fields
3. Normalize the policy weights
4. Compute the score and denoise
5. **Repeat while diffusion levels remain**
6. Execute, observe and start a new call

If reverse levels remain, draw new parameters from the updated noisy vector. Only after the final level can this planning call proceed to execution.

$$i > 1: \text{ sample again at level } i-1; \qquad i = 1: \text{ finish optimization}$$

Schematic algorithm walkthrough. The weighted mean updates MBD parameters; the selected evaluated policy is used for physical execution.

Archived method · prior formulation and results

Finish optimization before physical execution

1. **Read current information:** robot state, clarity, wind belief, geometry and target clarity.
2. **At each diffusion level:** sample nominal policy parameter vectors.
3. **Evaluate each vector:** predict applied inputs, motion and clarity in possible wind fields. For anticipated learning, update the predicted belief when observations become available.
4. **Refine:** average complete learning-updated monitoring costs, compute parameter weights and apply the MBD score update. Repeat only while diffusion levels remain.
5. **Execute:** after the final level, select and check the lowest-cost evaluated vector; apply its policy for the next execution interval.
6. **Replan:** assimilate real measurements and start a new planning call from the measured state and belief.

End of method. The following comparison concerns MBD; concrete vehicle models and numerical settings are in the appendix.

Archived method · prior formulation and results

5 Comparison with MBD

Retain the model-based score estimator; change what the model evaluates.

Algorithm comparison → unchanged components and method-specific components

Archived method · prior formulation and results

Algorithm comparison: what changes inside MBD?

Pan et al.: trajectory optimization

1. Initialize noisy trajectory vector $Y_N \sim \mathcal{N}(0, I)$.
2. For $i = N, \ldots, 1$, draw Gaussian decision-vector samples using the Gaussian proposal defined in the MBD preliminaries.
3. Roll out the control component to obtain dynamically feasible trajectories.
4. Compute task cost J and constraint indicator p_g.
5. Weight decision-vector samples by $p_g \exp(-J/\lambda)$.
6. Estimate the score and update $Y_i \mapsto Y_{i-1}$.

Wind-aware / anticipated-learning MBD

1. Initialize by adding forward-process noise to an evaluated initial policy parameter vector θ_{init}.
2. For $i = N, \ldots, 1$, sample policy parameter vectors using the same Gaussian rule.
3. **Predict the applied inputs, motion and clarity under wind. Proposed also updates the belief.**
4. Average complete learning-updated costs: $\hat{J}^{\text{learn}} = \hat{J}^{\text{fixed}} - \widehat{V}^{\text{net}}$.
5. Weight valid, in-domain **nominal parameters** by $\exp(-\hat{J}/\lambda)$.
6. Apply the same score update; retain the lowest-cost evaluated vector for checking and execution.

Adapted from Pan et al., Algorithm 2 with the proposal written consistently with Eq. (8). N is the number of reverse levels; Y is the paper's trajectory variable.

Archived method · prior formulation and results

Relationship to the MBD paper

Component	Pan et al.: MBD	This method
Decision being optimized	Trajectory / control representation	Parameters of a nominal control policy
What the model predicts	Dynamically feasible trajectory	Wind-dependent executed motion and sensing
Cost used by the score	Task cost; feasibility factors enter the density	Cumulative executed clarity deficit
Information inside the rollout	Task-dependent model state	Robot state, clarity and current or predicted wind belief
Reverse refinement	Model-based score and MCSA	Same construction applied to nominal parameters

The first contribution is disturbance-aware task evaluation. Anticipated learning extends that evaluation to changing beliefs.

Pan et al., Sections 4.1–4.2. The short numerical search is an approximation, not a new convergence guarantee.

Archived method · prior formulation and results

6 Results

Compare delivered monitoring in matched physical environments.

Wind-blind → wind-aware → wind-aware with anticipated learning

Archived method · prior formulation and results

Three levels of planning awareness

Question asked during planning	Wind-blind	Wind-aware	Proposed
What motion is nominally informative?	Yes	Yes	Yes
How will wind and the execution controller alter that motion?	Not predicted	Predicted from current wind belief	Predicted from current wind belief
How will future observations change later motion?	Not predicted	Belief held fixed	Belief updated during prediction

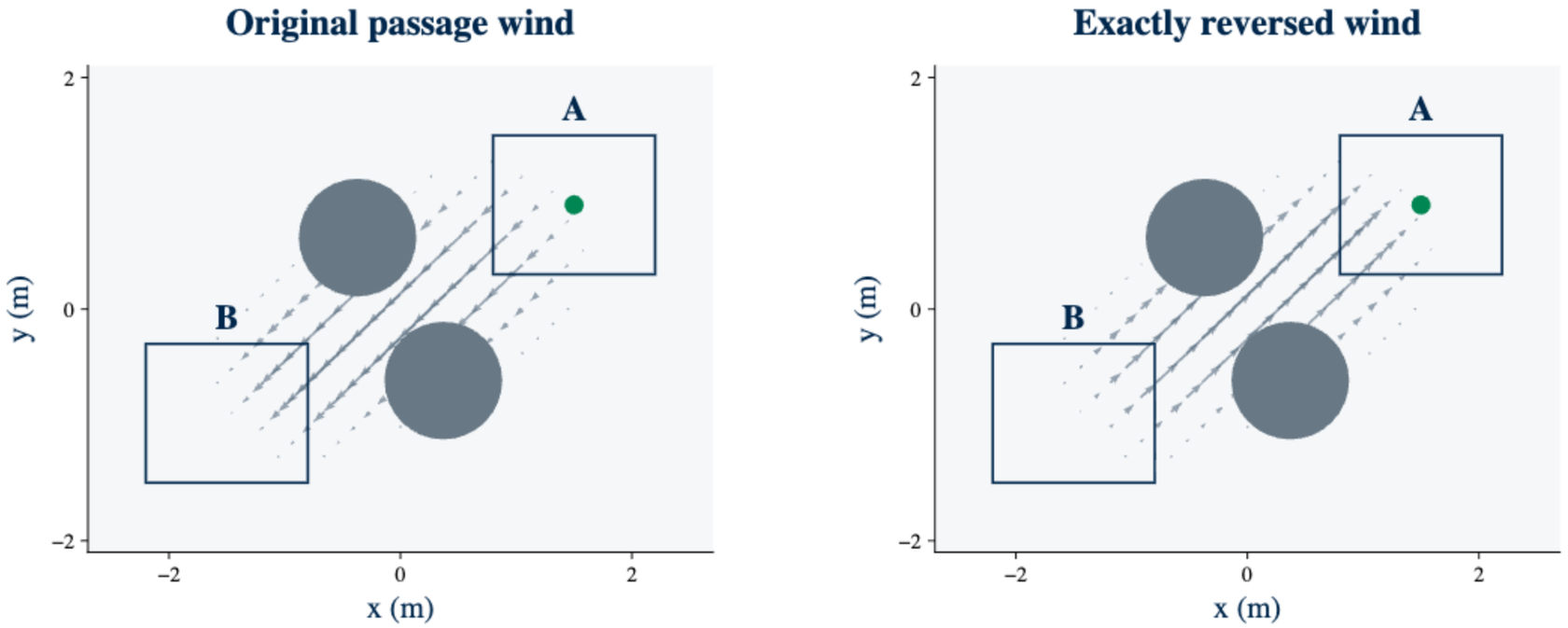
First contribution: evaluate monitoring on predicted executed motion.

Extension: account for observations that change later execution.

All three use the wind-informed controller during real execution. The archived wind-blind baseline also omits nominal wind compensation.

Archived method · prior formulation and results

Two environments with opposite wind directions

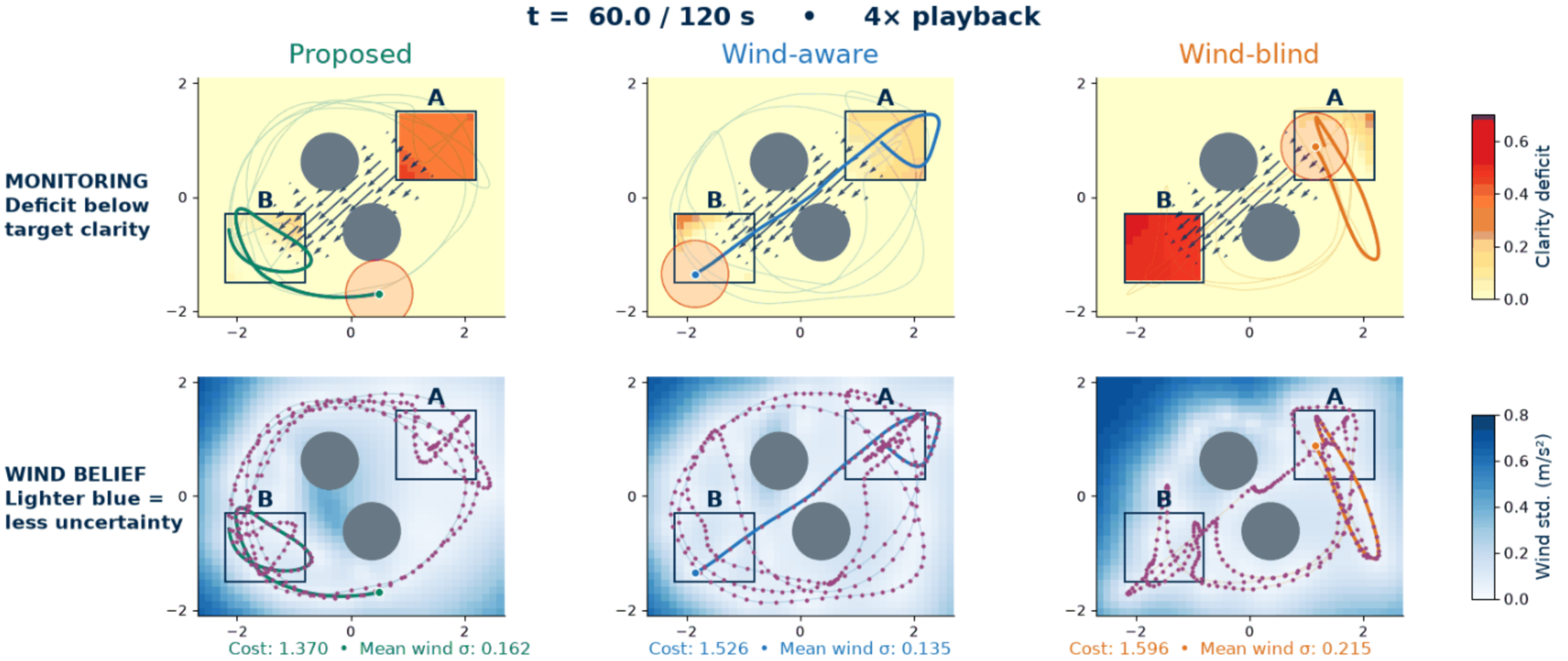


120 s missions · **10 s** planning horizon · **2 s** replanning

Matched initial state, clarity, task, survey and physical wind within each environment. Archived development runs; these predate the revised bounded MBD target. The signed-value equations decompose the chronological objective; no new mission is implied.

Archived method · prior formulation and results

Environment 1: original A/B-region wind

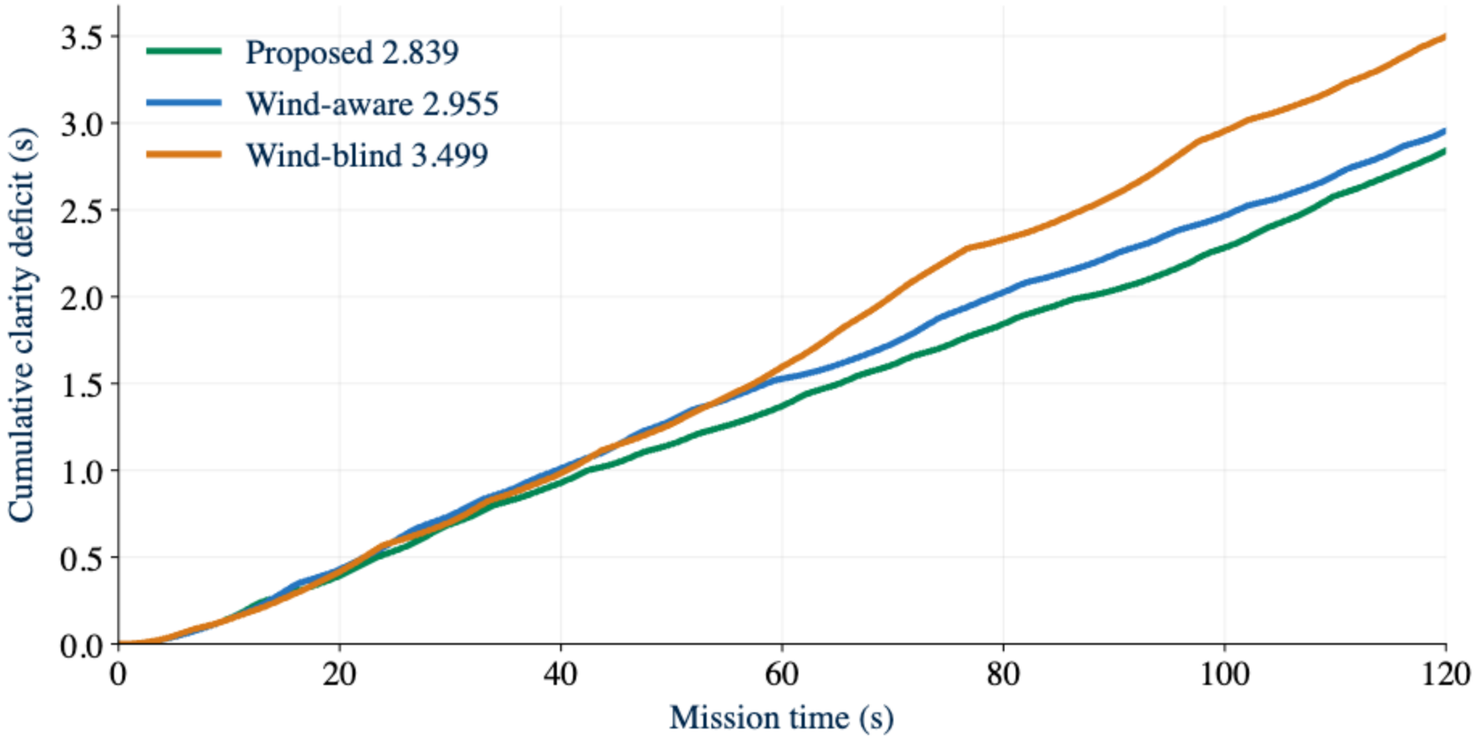


Proposed: **18.8% lower cost** than wind-blind; **3.9% lower** than wind-aware.

Top: monitoring deficit and wind arrows. Bottom: recorded wind uncertainty and observed locations. Fixed scales throughout.

Archived method · prior formulation and results

Original wind: compare delivered monitoring



Integrated clarity deficit

2.839 Proposed

2.955 Wind-aware

3.499 Wind-blind

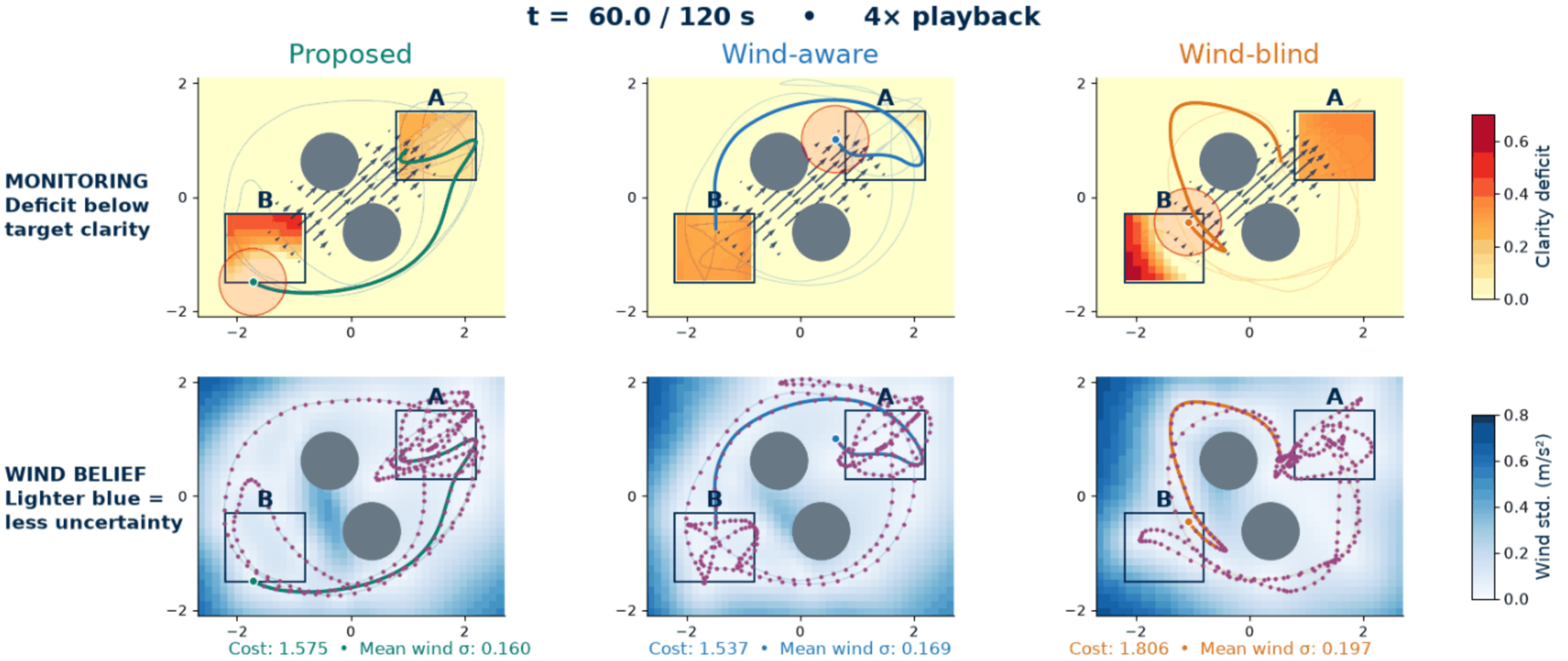
Completed A/B transfers

16 · 14 · 8, respectively

No physical contact in these three archived runs' dense audits.

Archived method · prior formulation and results

Environment 2: reversed wind

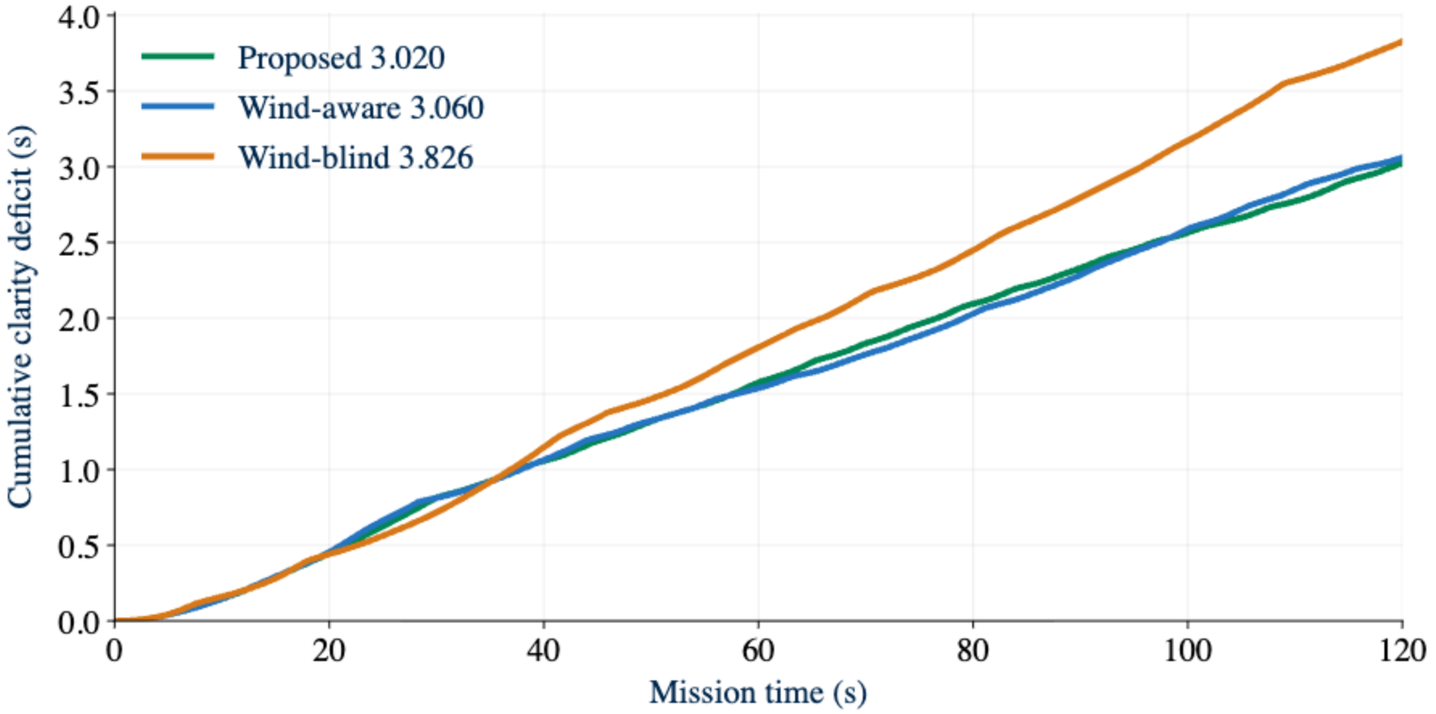


Proposed: **21.1% lower cost** than wind-blind; **1.3% lower** than wind-aware.

The entire physical wind field is reversed. Both color scales match Environment 1.

Archived method · prior formulation and results

Reversed wind: the additional learning gain is smaller



Proposed passage use

- Original wind: one middle crossing, A to B.
- Reversed wind: one middle crossing, B to A.
- Fifteen other transfers use bypasses in each run.

The observed direction changes. Consistent exploitation of the middle passage is not established.

Archived method · prior formulation and results

Separate the wind-aware gain from the learning gain

Recorded cost reduction	Original wind	Reversed wind
Wind-aware vs wind-blind	**15.5%**	**20.0%**
Proposed vs wind-aware	3.9%	1.3%
Proposed vs wind-blind	18.8%	21.1%

First contribution: predicting wind-dependent execution gives the larger contrast in these environments.

Extension: anticipating learning adds a smaller improvement in the displayed runs.

Descriptive comparisons from archived development runs. The wind-blind contrast also includes nominal wind compensation.

Archived method · prior formulation and results

What the current evidence establishes

Supported by these runs

- Wind-aware planning improves recorded monitoring over the historical wind-blind baseline.
- Anticipated learning adds a smaller improvement in both primary environments.
- These two archived missions do not establish a general learning advantage.

Still to establish

- A consistent anticipation advantage across independent physical environments.
- Full-mission performance of the revised numerical implementation.
- Reliability across larger search budgets and richer policy parameterizations.

Method correctness, optimizer quality and experimental performance are separate questions.

Archived method · prior formulation and results

Takeaways

1 **Plan for delivered monitoring.**
Use wind and the execution controller to predict where the robot can sense.

2 **Anticipate information through its effect on later motion.**
Update the predicted belief at scheduled observation times and continue the rollout.

3 **MBD optimizes this composed task model.**
The core score construction is retained; the policy evaluation becomes wind- and belief-aware.

Archived method · prior formulation and results

A Implementation appendix

Concrete simulation choices, kept separate from the generic method.

Policy representation → planar controller → numerical budgets → covariance-only variant

Archived method · prior formulation and results

Implementation: the 18-dimensional policy representation

A reference trajectory specifies desired positions for the tracking controller. The implementation uses $\theta = (a_0, a_1, \xi_1, \ldots, \xi_4, \operatorname{vec} G) \in \mathbb{R}^{18}$.

4 $a_0, a_1 \in \mathbb{R}^2$
Initial acceleration parameters

8 $\xi_1, \ldots, \xi_4 \in \mathbb{R}^2$
Future reference-position parameters

6 $G \in \mathbb{R}^{2\times 3}$
Belief-response coefficients

1 Initial interval: 0–2 s

Two acceleration parameters

$a_0, a_1 \in \mathbb{R}^2$

Interpolate nominal acceleration.

→

2 Future interval: 2–10 s

Four position parameters $\xi_1, \ldots, \xi_4$

Belief-response matrix $G \in \mathbb{R}^{2\times 3}$

Construct a cubic reference trajectory.

→

3 Nominal input

Track the reference.

Compensate estimated wind.

Nominal control input

Robot motion is computed afterward from the modified input and disturbed dynamics.

Bounded mappings convert the parameters to accelerations and reference positions. G modifies future reference positions using wind-belief features, defined below.

Archived method · prior formulation and results

Implementation: a planar tracking controller

The planar simulation specializes the generic robot model to

$$x = (r, v), \qquad \dot{r} = v, \qquad \dot{v} = u - \gamma v + d(r).$$

For the future reference r_{ref}:

$$u_{\text{nom}} = \text{clip}_{\mathcal{U}}\left(\ddot{r}_{\text{ref}} + k_p(r_{\text{ref}} - r) + k_v(\dot{r}_{\text{ref}} - v) + \gamma v - \mu_d(r)\right).$$

$r, v \in \mathbb{R}^2$: position and velocity.
γ: known linear drag coefficient.
$k_p, k_v > 0$: tracking gains.
Dots on r_{ref} denote time derivatives.

$\mu_d(r)$: mean of the available wind belief.
$\text{clip}_{\mathcal{U}}$: componentwise actuator saturation.
$\gamma v - \mu_d(r)$ compensates drag and estimated wind.

> **Nominal wind compensation uses the estimated field. Predicted robot motion uses a possible physical field drawn from that belief.**

The initial 2 s use the interpolated acceleration parameters instead of the reference-tracking terms. Defaults: $\gamma = 1$, $k_p = 1.2$, $k_v = 1.8$.

Archived method · prior formulation and results

Implementation: the velocity-damping design policy

To predict the implemented control modification, use the geometric constraint h_a from the preliminaries and a fixed velocity-damping design policy:

$$\pi_b(x; \mathcal{B}^d) = \mathrm{clip}_{\mathcal{U}}\left((\gamma - k_b)v - \mu_d(r)\right), \qquad k_b > 0.$$

π_b: design policy used to evaluate future constraint values.
k_b: velocity-damping gain; $k_b = 2$ here.

π_θ: nominal policy optimized for monitoring.
π_b is fixed and is not optimized by MBD.

Under mean wind and without saturation,

$$\dot{v} = (\gamma - k_b)v - \mu_d(r) - \gamma v + \mu_d(r) = -k_b v.$$

> **The design-policy prediction measures how current velocity and uncertain wind affect future constraint violation.**

Construction inspired by Knoedler et al.'s policy-value approach. It supplies an approximate execution model within our planning objective.

Archived method · prior formulation and results

Implementation: the planar input inequality

For $x = (r, v)$, partition the value gradient into

$$g_{r,a} = \nabla_r \widehat{V}_a, \qquad g_{v,a} = \nabla_v \widehat{V}_a.$$

The implemented inequality for obstacle or boundary a becomes

$$g_{v,a}^{\mathsf{T}} u \le -g_{r,a}^{\mathsf{T}} v + \gamma g_{v,a}^{\mathsf{T}} v - \max_{\ell} g_{v,a}^{\mathsf{T}} \tilde{d}^{[\ell]}(r) - \kappa \widehat{V}_a.$$

Through the design-policy prediction:
The wind belief changes the sampled trajectories, their largest constraint values $\widehat{V}_a$, and the gradients $g_{r,a}, g_{v,a}$.

Through instantaneous acceleration:
The largest projected wind acceleration among the inner samples changes the right-hand side of the input inequality.

> **These changes alter the predicted control input, motion and sensing. The monitoring objective accounts for all three effects.**

This row is obtained by substituting $\dot{r} = v$ and $\dot{v} = u - \gamma v + \tilde{d}^{[\ell]}(r)$ into the preceding rate condition.

Archived method · prior formulation and results

Implementation: belief-dependent reference positions

The matrix G in the policy parameter vector maps wind-belief features to a reference-position adjustment.

$$z(\mathcal{B}^d) = \frac{1}{s_d}\begin{bmatrix}\mu_{d,1}(r_*)\\ \mu_{d,2}(r_*)\\ \sigma_{d,1}(r_*)\end{bmatrix}, \qquad \Delta r(\mathcal{B}^d) = \delta_{\max}\tanh\left(Gz(\mathcal{B}^d)\right)$$

r_*: fixed central probe location.
$\mu_{d,1}, \mu_{d,2}$: two predicted wind components.
$\sigma_{d,1} = \sqrt{[\Sigma_d(r_*, r_*)]_{11}}$: wind standard deviation.

s_d: wind scale used to normalize features.
$\delta_{\max} = 0.6\,\mathrm{m}$: maximum reference shift.
The shift deforms future knots; cubic interpolation creates a reference trajectory.

This parameterization can deform the reference trajectory. It does not represent every possible future route switch.

Archived method · prior formulation and results

Implementation: a bounded target for parameter search

Set the unperturbed decision vector to $y = \theta$ and evaluate its **cumulative clarity deficit** $\hat{J}(\theta)$.

$$p_0(\theta) = \frac{1}{Z_\lambda}\, \mathbf{1}_\Theta(\theta)\, \mathbf{1}_\mathcal{V}(\theta) \exp\left(-\frac{\hat{J}(\theta)}{\lambda}\right)$$

$\Theta = [-6, 6]^{18}$: policy parameter domain in the revised implementation.
$\mathcal{V}$: policy parameter vectors with finite cost and no detected sampled contact.

$\mathbf{1}_A$: one when its argument belongs to set A, zero otherwise.
Z_λ: integral of the numerator; positive when valid finite-cost support has positive volume.

$$\hat{J}(\theta) = \frac{1}{S}\sum_{s=1}^{S} J_s(\theta) \quad \Longrightarrow \quad w_k \propto \exp\left(-\frac{\hat{J}(\theta^{[k]})}{\lambda}\right)$$

Average physical-field costs first. Use those costs to weight nominal parameters.

The executed controls and paths are outputs of the rollout. They are not substituted for $\theta^{[k]}$ in the weighted mean.

Archived method · prior formulation and results

Appendix: covariance-only anticipation

For fixed sensing locations $\mathcal{R}$ and fixed GP hyperparameters, one can predict covariance reduction:

$$\Sigma_d^+(r, r') = \Sigma_d(r, r') - \Sigma_d(r, \mathcal{R})\left[\Sigma_d(\mathcal{R}, \mathcal{R}) + R_{\mathcal{R}}\right]^{-1}\Sigma_d(\mathcal{R}, r').$$

$\mathcal{R}$: predicted executed measurement locations.
$R_{\mathcal{R}}$: observation-noise covariance for that batch.
Σ_d^+: covariance after the hypothetical batch.

Keep the predicted mean at μ_d and rebuild the execution model using Σ_d^+.
This approximates learning without drawing measurement values.

The mean-fixed approximation captures changing uncertainty, but omits adaptation to different observed wind means.

This is a separate implementation mode. The result animations show the realization-based proposed method.

Archived method · prior formulation and results

Appendix: sources and reproducibility

Problem and method

- Naveed, *Informative Planning with Task-Aware Spatial Disturbance Estimation*: supplied manuscript, problem notation and clarity.
- Pan et al., *Model-Based Diffusion for Trajectory Optimization* (2024), Eqs. (6), (8)–(10), Algorithm 2. Paper
- Chen, Liu and Khardon, *POAM* (RSS 2024). Paper
- Knoedler et al., *Safety on the Fly* (RA-L 2025): robust policy control barrier functions.

Reproducible presentation

- Exact archived run paths and hashes: `results_manifest.json`.
- GIFs use recorded motion, clarity and wind-belief histories with fixed color scales.
- Editable Quarto source, LaTeX math, local KaTeX fonts and vector architecture are included.

Archived method · prior formulation and results